



TRABAJO FIN DE GRADO

GRADO EN DESARROLLO DE APLICACIONES MULTIPLATAFORMA

CURSO ACADÉMICO 2025/2026

CONVOCATORIA OCTUBRE 2025(14/10/2025)

ILERNA SMART

AUTOR(A): Fernández de Gomar , Antonio David

DNI: 20602262N

GITHUB: <https://github.com/antoniodavid13>

En Madrid, a 14 de Octubre de 2025

RESÚMENES

Resumen

La aplicación Ilerna Smart está pensada como una solución al e-learning y gestión académica de manera moderna y accesible dirigida a los alumnos y profesores de la institución de formación específica Ilerna. Está desarrollada con un frontend en React y Kotlin para una experiencia de usuario moderna, estable y accesible y con un propósito de ser multiplataforma, y un backend robusto con arquitectura de microservicios en Java junto con Spring Boot, su principal valor reside en la integración de la IA en los procesos de enseñanza, evaluación y mejora. El sistema estructura las formaciones por asignaturas, permitiendo así a los alumnos acceder a resúmenes dinámicos generados por la IA Grok de cada documentación adjuntada en los temas para un estudio eficiente y centrado.

El componente central de la aplicación es la integración de la IA en los procesos donde recae en la AI Grok usado a partir de una api key, es elegida debido a que permite tener una cuota diaria generosa en la fase de prueba alcanzado 30/min, facilitando así la posibilidad de realizar demos funcionales previas sin ningún coste económico extra a la configuración definitiva de producción. Gracias a la IA Grok se pueden realizar procesos automatizados como la creación de preguntas de tests a través de la IA mediante un prompt y un System Message completamente personalizado para cumplir con el contexto buscado, la recogida de las calificaciones del alumno y el procesamiento de los resultados. Además, el sistema detalla y resalta los errores del estudiante, almacenando todo en el módulo de calificaciones para permitir la revisión posterior del test y asegurar una evaluación constante.

Simultáneamente, la IA de Grok envía un informe personalizado al alumno donde se detallan las preguntas falladas, junto con un resumen y una explicación específica del tema que se necesita repasar.

Abstract

The Ilerna Smart application is designed as a comprehensive e-learning and academic management solution tailored for students and teachers within a specific training institution. Developed with a frontend in React and Kotlin to ensure a modern, stable, and accessible user experience, and a robust backend powered by a microservices architecture using Java and Spring Boot, its core value lies in the integration of Artificial Intelligence within the teaching

and assessment processes. The system organizes course content by subjects, allowing students to access dynamic summaries of all documentation attached to the topics, promoting an efficient and focused study method.

The central component of this AI integration relies on the Grok API, which was selected because it offers a generous daily quota during the testing phase, reaching up to 30 requests per minute. This technical advantage facilitates the deployment of fully functional demos prior to the final production setup. Through this API, several automated processes are executed, such as the generation of quiz questions using customized prompts and system messages, as well as the collection of student grades and result processing. Furthermore, the system details and highlights the student's specific mistakes, storing all data within the grading module to allow for subsequent test reviews and ensure continuous evaluation.

Simultaneously, Grok's AI sends a personalized report to the student detailing the failed questions, alongside a targeted summary and a specific explanation of the topics that require further review. This automated workflow optimizes the correction process and grade storage, transforming Ilerna Smart into a powerful tool for directed learning, reduction of teacher workload, and continuous academic improvement for the student body.

1 INTRODUCCIÓN

La irrupción de Internet en la década de 1990 supuso una transformación profunda en los medios de comunicación y marcó un antes y un después en el ámbito educativo. La convergencia entre las telecomunicaciones y el procesamiento de datos dio lugar al concepto de telemática, permitiendo un intercambio de información más rápido, accesible y globalizado. Este avance tecnológico sentó las bases para el desarrollo de las primeras plataformas educativas, páginas web con recursos didácticos y modelos iniciales de formación en línea (Duarte-Gastélum, 2024).

Desde entonces, la evolución de las tecnologías de la información y la comunicación (TIC) ha modificado de manera significativa tanto la educación como la interacción humana. La integración de herramientas digitales en los entornos académicos ha transformado la planificación educativa, las metodologías docentes y las dinámicas de aprendizaje, consolidando la innovación tecnológica como un elemento indispensable para el desarrollo social y humano. En un contexto globalizado y en constante cambio, las instituciones

educativas se enfrentan al reto de adaptarse a nuevas necesidades formativas y perfiles de estudiantes cada vez más digitales (Mendoza y Torres-Zapata, 2024).

Este desarrollo tecnológico ha obligado al profesorado a abandonar progresivamente los modelos tradicionales de enseñanza para incorporar nuevas metodologías apoyadas en las TIC. La rápida evolución de las herramientas digitales exige a los docentes una actualización constante de sus competencias pedagógicas y tecnológicas, así como la creación de espacios de aprendizaje más interactivos, colaborativos y dinámicos. Paralelamente, el alumnado también necesita desarrollar habilidades digitales y comunicativas que le permitan desenvolverse de manera eficiente dentro de entornos virtuales de aprendizaje (Mendoza y Torres-Zapata, 2024).

En este escenario, la educación online se ha consolidado como una alternativa clave para la formación académica y profesional. Su flexibilidad y accesibilidad han impulsado a numerosas instituciones educativas a adoptar modelos virtuales capaces de optimizar recursos y ampliar el alcance de la enseñanza. Además, en un mercado laboral en constante evolución, la formación digital favorece el desarrollo de competencias profesionales y tecnológicas altamente demandadas. La educación superior online también se proyecta como una vía hacia la internacionalización del aprendizaje mediante iniciativas como los cursos masivos abiertos en línea (MOOC), facilitando el acceso al conocimiento desde cualquier parte del mundo (Garcés y Bastías, 2025).

Dentro de este proceso de transformación digital, instituciones educativas como Ilerna han adquirido un papel relevante al desarrollar plataformas orientadas a la formación telemática y digital de estudiantes de Ciclos Formativos de Grado Superior. Estas plataformas permiten adaptar los contenidos educativos a entornos virtuales, favoreciendo un modelo de aprendizaje flexible y accesible.

Sin embargo, a pesar de los avances que ha supuesto la educación a distancia, este modelo también presenta importantes desafíos. Uno de los principales problemas actuales es la escalabilidad del sistema educativo online y la necesidad de mantener una comunicación constante, clara y efectiva entre docentes y estudiantes. En este contexto, la retroalimentación docente adquiere un papel fundamental, ya que no debe limitarse únicamente a la evaluación, sino actuar como una guía que oriente al alumnado sobre sus objetivos, su progreso y los siguientes pasos en el proceso de aprendizaje (Camian, 2023).

Asimismo, la transición desde la enseñanza presencial hacia entornos virtuales requiere una adaptación profunda tanto por parte del alumnado como del profesorado. Las metodologías tradicionales no siempre pueden trasladarse directamente al ecosistema digital, lo que obliga a desarrollar nuevas estrategias pedagógicas y competencias tecnológicas. La falta de formación específica en metodologías online puede generar frustración, desmotivación y dificultades en el proceso educativo, problemas que frecuentemente se atribuyen erróneamente a la tecnología cuando, en realidad, derivan de una insuficiente preparación de los agentes implicados (Borges, 2005).

En este contexto de transformación educativa y tecnológica surge la necesidad de desarrollar herramientas innovadoras capaces de mejorar la experiencia de aprendizaje y optimizar la interacción entre alumnado y profesorado. Es aquí donde aparece Ilerna Smart, una aplicación basada en inteligencia artificial orientada a apoyar la docencia y facilitar el acceso personalizado al conocimiento dentro de los entornos educativos digitales.

1.1 JUSTIFICACIÓN

Debido a la situación actual de la educación telemática se ha podido comprobar que existe una brecha entre el ideal y la realidad, Según Guerrero et al.(2020; citados en Vallejo, 2022) la enseñanza virtual "es una modalidad pedagógica o didáctica que incrementa la calidad del proceso de aprendizaje de los estudiantes porque respeta la flexibilidad o accesibilidad, es decir, puede dirigirse en fases y espacios cambiantes". Este problema no solo está en Ilerna si no que la mayoría de educaciones online.

Este problemático suceso se manifiesta principalmente en tres justificaciones principales por el cual se ha desarrollado Ilerna Smart:

- **Orientación al Alumnado:** Según Vallejo (2022) a pesar de su importancia crítica, la orientación integral de los estudiantes hacia el aprendizaje en línea sigue siendo uno de los mayores problemas sin resolver en la educación virtual actual, ya que las instituciones tienden a favorecer la matriculación masiva por razones económicas en lugar de asesorar y regular la carga académica real del estudiante. Una vez iniciado el curso, esta desventaja se ve agravada por la falta de apoyo estructurado al aprendizaje; En lugar de guiar activamente al estudiante en su primer contacto con el entorno digital y planificar su tiempo, existe una preocupante falta de presencia regular en las aulas virtuales, donde el feedback continuo ha sido sustituido

por calificaciones automatizadas que no actúan como la guía necesaria para aclarar los objetivos y mantener motivados a los estudiantes..

- Dificultades en la relación estudiante - docente : Según Díaz y Maldonado (2013) la transición a un entorno virtual (e/b/m-Learning) introduce una barrera crítica, transformando las interacciones educativas en comunicación impersonal, lo que degrada gravemente la experiencia del alumno y el proceso de aprendizaje. Al eliminar el contacto físico, el entorno digital cierra por completo la comunicación no verbal, dejando a la imaginación los matices esenciales del lenguaje corporal, los gestos y la entonación que humanizan el aprendizaje en un aula tradicional.
- Sistema educativo Español: El sistema educativo en España presenta críticos problemas estructurales en la enseñanza, un sector en crisis, donde la carga de trabajo administrativo y la dificultad de gestionar unas aulas con una creciente diversidad cultural, social y académica están obligando a muchos docentes a abandonar sus puestos de trabajo. Esta falta de formación específica para resolver los conflictos escolares y el desgaste general crea una visión deprimente del relevo generacional: las duras condiciones laborales y el declive del prestigio de la profesión hacen que los estudiantes ya no quieran ser docentes. Esta preocupante realidad limita el tiempo de preparación de las lecciones y la atención personalizada, precisamente cuando la evidencia científica enfatiza que una gestión eficaz del aula y profesores motivados son factores clave para mejorar el rendimiento académico y garantizar un ambiente de aprendizaje adecuado (Tahull y Molina-Luque, 2025).

1.2 MOTIVACIÓN

Ilerna Smart como proyecto para trabajo final de grado es ambicioso ya que se busca lograr una convergencia entre el campus de Ilerna y Ilerna Smart, debido a ese objetivo hace que la complejidad de la aplicación aumente tanto como el número de entidades relacionadas como el número de procesos y servicios personalizados que tiene cierta coherencia con el campus.

Los pilares que impulsan la realización de este trabajo se articulan en los siguientes puntos:

- Herramientas asistidas orientadas a la docencia: A nivel profesional orientado a la docencia, existe una motivación en diseñar soluciones tecnológicas que alivien la carga

cognitiva del profesorado. El hecho de desarrollar una aplicación con la capacidad de dar soporte a cientos de profesores permitiendo así la posibilidad de gestionar grupos masivos de estudiantes de manera controlada hace que sea uno de los motores de esta investigación .

- **Desafío en el desarrollo de software escalable:** Desde una perspectiva técnica y profesional, el diseño de un servicio que esté activo en tiempo real permitiendo así la calificación y visualización de las calificaciones de los estudiantes a partir de una IA, hace que el proyecto tenga una dificultad técnica elevada teniendo que implementar la arquitectura de software: microservicios con el fin de que aplicación tenga un rendimiento adecuado y no se sature, la motivación no está solo en el simple hecho de una desafío técnico sino en una proyección orientada a la realidad de los estudiantes que una vez graduados se tengan que enfrentar.
- **Impacto en la institución educativa:** Más allá de la eficiencia técnica, si gracias a este proyecto se logra conseguir la mejora de calidad educativa de Ilerna Online, los futuros estudiantes formados a partir de este sistema tendrán una mejor formación académica y estarán más preparados para la vida laboral, por lo tanto tendrán mayores salidas laborales y con ilusiones de seguir formándose en este sector, esto supone una gran motivación de que gracias a una herramienta creada inicialmente por un estudiante del mismo centro educativo se forje el sueño de muchos otros.
- **Cierre de ciclo formativo:** Por último, este trabajo representa el clímax de conocimiento alcanzado durante el grado, integrando diversas disciplinas tecnológicas en un producto final que posee una aplicación práctica inmediata en instituciones educativas modernas.

1.3.- RESUMEN DE LOS CAPÍTULOS DEL TRABAJO

A continuación, se detalla la estructura y el contenido de los capítulos que conforman el presente Trabajo de Fin de Grado, los cuales han sido organizados de manera lógica para facilitar la comprensión del desarrollo y los resultados del proyecto:

- **Capítulo 1. Introducción:** Se establece el marco inicial del trabajo, detallando los motivos que impulsan el desarrollo de la aplicación, la justificación de su necesidad en el contexto educativo actual y una breve evolución histórica de la educación digital.

- Capítulo 2. Estado del arte: Representa el núcleo teórico del trabajo. En esta sección se analiza la literatura académica relevante mediante la revisión de artículos especializados y se describen las metodologías, arquitecturas y herramientas de software que sirven de base para el desarrollo de Ilerna Smart.
- Capítulo 3. Objetivos del trabajo: Se definen de forma precisa el objetivo general y los subobjetivos específicos que se pretenden alcanzar, estableciendo los criterios de éxito para la implementación de la solución propuesta.
- Capítulo 4. Metodología: Se detalla el procedimiento y el marco de trabajo seleccionado para la ejecución del proyecto, justificando la elección de metodologías existentes que aseguren un desarrollo eficiente y profesional.
- Capítulo 5. Contribución del trabajo: Este capítulo expone el valor original del proyecto. Incluye el desarrollo de la solución técnica, la explicación teórica de su funcionamiento y el caso de estudio donde se pone a prueba la herramienta.
- Capítulo 6. Resultados y evaluación: Se presentan los datos obtenidos tras la implementación del caso de estudio, analizando el rendimiento de la aplicación y el cumplimiento de las expectativas iniciales.
- Capítulo 7. Conclusiones y trabajo futuro: Se sintetizan los hallazgos principales y se reflexiona sobre los objetivos alcanzados. Asimismo, se proponen líneas de investigación y desarrollo para futuras versiones o mejoras del sistema.
- Capítulo 8. Bibliografía: Recopila todas las fuentes documentales, artículos científicos y recursos técnicos consultados durante la elaboración del trabajo, debidamente citados bajo la normativa APA.

2.0 ESTUDIO DEL ARTE

2.1 Inteligencia Artificial Generativa aplicada a la Evaluación

La inteligencia artificial (IA) se ha convertido en una de las tecnologías con mayor capacidad de transformación dentro del ámbito educativo contemporáneo. Su evolución durante los últimos años ha permitido el desarrollo de sistemas capaces de automatizar procesos, analizar grandes volúmenes de información y ofrecer experiencias de aprendizaje más personalizadas y dinámicas. Actualmente, la IA puede aplicarse a numerosos campos académicos, incluyendo las ciencias sociales, la educación en ingeniería, la educación científica, la educación médica, las ciencias de la vida y el aprendizaje de idiomas, entre otros (Hadzhikoleva et al., 2024).

Dentro de este contexto, la inteligencia artificial generativa representa uno de los avances tecnológicos más relevantes en el ámbito educativo. Este tipo de IA se define como un conjunto de modelos complejos capaces de generar contenido similar al producido por humanos, incluyendo textos, respuestas contextualizadas, análisis, imágenes y materiales didácticos (Hernández, 2024). Gracias a estas capacidades, la IA generativa está transformando progresivamente los modelos tradicionales de enseñanza y aprendizaje.

La incorporación de inteligencia artificial generativa en educación está provocando cambios estructurales en todos los niveles educativos. Estas herramientas permiten automatizar la creación de recursos de aprendizaje, adaptar contenidos según las necesidades individuales del alumnado y optimizar procesos relacionados con la evaluación y el seguimiento académico (Hernández, 2024). Como consecuencia, las instituciones educativas comienzan a integrar sistemas inteligentes dentro de sus plataformas digitales y entornos virtuales de aprendizaje.

La expansión de la educación online y el incremento del número de estudiantes en plataformas virtuales han generado importantes desafíos relacionados con la atención individualizada y la evaluación continua. Ante esta situación, diferentes investigaciones han impulsado el desarrollo de programas basados en inteligencia artificial capaces de responder a estas limitaciones y mejorar la eficiencia de los procesos educativos (An et al., 2021; Erdemir, 2019; Leung et al., 2022; Pillay et al., 2018, citados en Hernández, 2024).

Uno de los principales beneficios de la IA aplicada a la educación es su capacidad para recopilar y procesar información en tiempo real. Gracias al uso de algoritmos avanzados y sistemas de aprendizaje automático, estas herramientas pueden analizar patrones de

comportamiento, detectar dificultades de aprendizaje y generar respuestas adaptadas al contexto educativo específico de cada estudiante.

Asimismo, la inteligencia artificial facilita la automatización de múltiples tareas académicas y administrativas. Entre ellas destacan la generación automática de cuestionarios, rúbricas, actividades y sistemas de evaluación personalizados, así como la creación de asistentes virtuales capaces de proporcionar apoyo continuo al alumnado (García-Peñalvo et al., 2023, citados en Hernández, 2024).

La retroalimentación inmediata constituye otra de las principales ventajas de la IA generativa dentro de los procesos educativos. Mediante modelos de procesamiento del lenguaje natural, los sistemas inteligentes pueden analizar respuestas y producciones textuales de los estudiantes para identificar errores, ofrecer sugerencias y proporcionar explicaciones adaptadas al nivel de comprensión del usuario.

Estas capacidades favorecen el desarrollo de modelos de aprendizaje más autónomos y personalizados. Además, permiten fomentar el pensamiento crítico del alumnado al ofrecer respuestas contextualizadas y orientación continua durante el proceso de aprendizaje (García-Peñalvo et al., 2023, citados en Hernández, 2024).

Sin embargo, diversos autores destacan que la inteligencia artificial debe utilizarse como una herramienta de apoyo complementaria y no como un sustituto del proceso educativo o de la figura del docente. Según Alíer et al. (2024), la IA generativa debe respaldar la experiencia de aprendizaje y potenciar las capacidades del profesorado en lugar de reemplazar completamente la intervención humana.

En este contexto, los grandes modelos de lenguaje (LLM) han adquirido una relevancia especialmente significativa dentro de los entornos educativos. Estos modelos poseen la capacidad de comprender lenguaje natural, interpretar contextos complejos y generar respuestas coherentes en tiempo real, convirtiéndose en herramientas altamente eficaces para la automatización de procesos educativos y evaluativos.

La creciente complejidad de los LLM ha impulsado su utilización en múltiples sectores, incluyendo el soporte técnico, la investigación científica y la educación (Edson y Weigang, 2025). Dentro del ámbito educativo, estos modelos permiten analizar grandes cantidades de

información y generar pruebas, diagnósticos y sistemas de evaluación adaptativa con elevados niveles de precisión técnica.

Uno de los ejemplos más representativos de esta evolución tecnológica es Grok, un sistema basado en inteligencia artificial generativa capaz de analizar conversaciones y producciones textuales utilizando modelos avanzados de procesamiento lingüístico (Wang y Cardeñosa, 2025). Gracias a ello, Grok puede evaluar parámetros como la semántica, el contexto, la coherencia gramatical y la coordinación lingüística de forma automatizada.

Este procesamiento automatizado de datos permite identificar niveles de comprensión y generar diagnósticos académicos precisos. Además, reduce considerablemente el tiempo destinado a tareas de corrección manual y facilita el desarrollo de modelos de evaluación continua dentro de entornos virtuales de aprendizaje.

La utilización de herramientas como Grok también permite establecer sistemas de corrección automatizada capaces de ofrecer una primera evaluación preliminar que posteriormente puede ser revisada y validada por el profesorado (Wang y Cardeñosa, 2025). Este enfoque combina automatización y supervisión humana, optimizando la eficiencia sin eliminar completamente el papel del docente.

Dentro del aprendizaje de idiomas, por ejemplo, un profesor puede diseñar escenarios conversacionales interactivos centrados en aspectos gramaticales concretos, como el uso del subjuntivo. A partir de las respuestas del estudiante, la IA analiza el contexto lingüístico y adapta dinámicamente las actividades según el progreso detectado (Wang y Cardeñosa, 2025).

Esta capacidad de personalización representa uno de los aspectos más relevantes de la inteligencia artificial aplicada a la educación. Según Alíer et al. (2024), la IA permite adaptar los procesos de aprendizaje a diferentes ritmos, necesidades y estilos cognitivos, favoreciendo una educación más inclusiva y centrada en el estudiante.

La arquitectura transformadora utilizada por modelos como Grok emplea técnicas de aprendizaje por transferencia y refinamiento adaptativo capaces de optimizar las interacciones contextuales y el análisis de información en tiempo real (Edson y Weigang, 2025). Gracias a ello, los sistemas inteligentes pueden adaptarse de forma dinámica al comportamiento y evolución del alumnado.

Además, la integración de Grok con flujos masivos de conversaciones y registros lingüísticos permite disponer de bases de datos amplias y diversas capaces de enriquecer los procesos de evaluación. Esto facilita la generación de pruebas dinámicas adaptadas a tendencias lingüísticas y contextos socioculturales contemporáneos (Edson y Weigang, 2025).

Los modelos de inteligencia artificial también están impulsando el desarrollo de sistemas de aprendizaje adaptativo y modelos predictivos capaces de analizar información académica y conductual del alumnado. Estas herramientas permiten detectar patrones de desmotivación, dificultades de aprendizaje y posibles situaciones de riesgo académico de forma temprana (Hadzhikoleva et al., 2024).

Gracias al procesamiento inteligente de datos, los sistemas educativos pueden estructurar experiencias de aprendizaje personalizadas y generar pruebas adaptativas capaces de modificar su dificultad según el rendimiento y evolución del estudiante (Hadzhikoleva et al., 2024).

La evaluación adaptativa supone un cambio significativo respecto a los modelos tradicionales basados en pruebas estandarizadas. En lugar de aplicar el mismo examen a todos los estudiantes, la IA permite construir evaluaciones dinámicas capaces de ajustarse automáticamente al nivel competencial de cada usuario.

Los docentes también pueden utilizar herramientas de IA para generar materiales educativos, proporcionar comentarios automáticos sobre tareas y diseñar actividades personalizadas según objetivos pedagógicos concretos (Hadzhikoleva et al., 2024). Esto mejora considerablemente la eficiencia de los procesos de planificación y seguimiento académico.

La integración de capacidades de IA dentro de sistemas de creación y gestión de pruebas automatizadas también optimiza la recopilación de información sobre el desempeño del alumnado. Estas herramientas permiten analizar diferentes habilidades cognitivas y generar pruebas altamente personalizables con distintos niveles de dificultad.

Además, los sistemas inteligentes pueden incorporar múltiples formatos de evaluación, incluyendo preguntas abiertas, ejercicios prácticos, cuestionarios de opción múltiple o ensayos automatizados. Esta diversidad favorece el desarrollo del pensamiento crítico y creativo del alumnado (Hadzhikoleva et al., 2024).

Tradicionalmente, la elaboración de pruebas convencionales requería una importante inversión de tiempo y recursos humanos, además de procesos complejos de validación y revisión (Hadzhikoleva et al., 2024). La automatización impulsada por inteligencia artificial surge como respuesta a esta problemática, permitiendo optimizar todo el ciclo de evaluación académica.

Los avances en procesamiento del lenguaje natural han permitido desarrollar modelos generativos entrenados con millones de parámetros y grandes volúmenes documentales (Hughes, 2023). Estas tecnologías son capaces de mantener conversaciones contextualizadas en tiempo real y generar respuestas con significado humano (Lopezosa, 2023).

Dentro de la evaluación educativa, estas capacidades permiten analizar simultáneamente información lingüística y conceptual para generar pruebas adaptadas al flujo de interacción del estudiante (Gutiérrez, 2023). De este modo, los sistemas inteligentes pueden modificar dinámicamente el nivel de dificultad y la complejidad de las preguntas según el rendimiento detectado.

No obstante, el uso de inteligencia artificial generativa en educación también presenta riesgos y limitaciones importantes. Según Delany (2024), la ausencia de supervisión adecuada y el uso incorrecto de estas herramientas pueden afectar la validez de los razonamientos y reducir el papel crítico del docente dentro del proceso educativo.

Esta problemática genera una nueva relación entre el estudiante, el profesorado y la inteligencia artificial. Si los docentes no desarrollan competencias relacionadas con el uso de IA y la creación de instrucciones adecuadas, existe el riesgo de adoptar una posición pasiva frente al avance tecnológico (Delany, 2024).

Como consecuencia, aumenta la necesidad de formar tanto a estudiantes como a profesores en competencias relacionadas con inteligencia artificial, alfabetización digital e ingeniería de *prompts*. Estas capacidades resultan fundamentales para garantizar la adaptación de la población a los nuevos entornos académicos y profesionales impulsados por la IA.

Diversos investigadores, como Margaret Boden, Ray Kurzweil y Eliezer Yudkowsky, han destacado la relevancia de la inteligencia artificial y del diseño de *prompts* como herramientas clave para el desarrollo tecnológico y humano futuro (Delany, 2024). Estas perspectivas

resaltan la importancia de comprender el funcionamiento de los sistemas inteligentes dentro de la sociedad contemporánea.

La comunicación con los modelos de lenguaje generativo se realiza mediante instrucciones en lenguaje natural denominadas *prompts*. Estas indicaciones constituyen el principal mecanismo de interacción entre el usuario y el modelo de IA, siendo fundamental su correcta formulación para obtener respuestas precisas y útiles (Quintero y German, 2025).

La ingeniería de *prompts* se ha convertido en una disciplina esencial dentro de los sistemas basados en inteligencia artificial generativa. Su objetivo consiste en optimizar las instrucciones proporcionadas a los modelos de IA para mejorar la calidad, precisión y coherencia de las respuestas generadas (Quintero y German, 2025).

En entornos educativos, los *prompts* funcionan como una interfaz semántica capaz de definir tanto el resultado esperado como el procedimiento lógico que debe seguir el sistema para generarlo. Gracias a ello, los docentes pueden estructurar actividades personalizadas y adaptar el comportamiento de los modelos inteligentes según objetivos pedagógicos específicos.

Aunque los sistemas de IA generativa han mejorado significativamente su capacidad para comprender preguntas complejas, la calidad de sus respuestas continúa dependiendo en gran medida de cómo se formulan las instrucciones proporcionadas por el usuario (Quintero y German, 2025). Por este motivo, el dominio de la ingeniería de *prompts* adquiere una importancia creciente dentro del ámbito educativo.

En este escenario de transformación tecnológica surge la necesidad de desarrollar plataformas educativas capaces de integrar inteligencia artificial generativa, sistemas adaptativos y herramientas de evaluación automatizada dentro de un entorno unificado. Bajo esta perspectiva aparece Ilerna Smart, una aplicación orientada a optimizar los procesos de enseñanza y evaluación mediante el uso de inteligencia artificial, aprendizaje personalizado y automatización inteligente. La plataforma busca mejorar la interacción entre alumnado y profesorado, facilitar la creación dinámica de contenidos y ofrecer experiencias educativas adaptadas a las necesidades específicas de cada estudiante dentro de entornos de formación online.

2.2 Sistemas de Tutorización Inteligente y Feedback Personalizado

Los Sistemas de Tutoría Inteligente (ITS, por sus siglas en inglés) representan una de las principales evoluciones tecnológicas dentro del ámbito de la educación digital. Estos sistemas están diseñados para ofrecer experiencias de aprendizaje personalizadas que imitan la orientación y atención individualizada de un docente humano (Wang et al., 2025). Su objetivo principal consiste en adaptar dinámicamente el proceso de enseñanza según las necesidades, capacidades y comportamiento de cada estudiante.

El desarrollo de los ITS ha estado estrechamente vinculado al avance de las tecnologías de aprendizaje automático (*Machine Learning*, ML). Gracias a la incorporación de estos algoritmos, los sistemas pueden analizar continuamente los patrones de respuesta y el comportamiento del usuario para adaptar el ritmo, el contenido y las estrategias pedagógicas en tiempo real (Wang et al., 2025).

La evolución de estos sistemas ha permitido superar las limitaciones de los modelos educativos tradicionales basados en estructuras rígidas y homogéneas. En lugar de aplicar un único itinerario formativo para todos los estudiantes, los ITS modernos ofrecen procesos educativos flexibles y personalizados que se ajustan constantemente al progreso del usuario.

En este contexto, los grandes modelos de lenguaje (LLM) han abierto nuevas oportunidades para mejorar los sistemas de tutoría inteligente. Gracias a sus capacidades de procesamiento del lenguaje natural, los LLM permiten ofrecer retroalimentación interactiva mediante interfaces conversacionales capaces de responder preguntas, resolver dudas y generar contenidos educativos de manera natural y fluida (Wang et al., 2025).

Las investigaciones actuales relacionadas con ITS basados en LLM demuestran que estos sistemas mejoran significativamente la interacción entre el estudiante y la plataforma educativa. La posibilidad de mantener conversaciones contextualizadas favorece una experiencia más cercana y personalizada, aumentando el compromiso y la participación del alumnado.

Uno de los principales problemas dentro de los entornos de formación online es la dificultad de los estudiantes para identificar qué competencias deben priorizar y cuál es la mejor ruta de aprendizaje para alcanzar sus objetivos académicos o profesionales. Esta

situación resulta especialmente frecuente en contextos tecnológicos y profesionales altamente cambiantes (Wang et al., 2025).

La ausencia de una orientación estructurada puede generar sobrecarga cognitiva y provocar que los estudiantes inviertan tiempo y recursos en formaciones poco relevantes o alejadas de las necesidades reales del entorno laboral (Wang et al., 2025). Como respuesta a esta problemática surge el enfoque de aprendizaje orientado a objetivos.

El aprendizaje orientado a objetivos va más allá de la simple transferencia de contenidos. Este modelo organiza todo el proceso formativo alrededor de competencias y metas específicas, alineando los recursos educativos con los resultados que el estudiante desea alcanzar (Wang et al., 2025).

Dentro de los sistemas ITS orientados a objetivos, las dinámicas pedagógicas se adaptan continuamente a las necesidades individuales del usuario. Estos sistemas diagnostican las carencias de habilidades del estudiante, analizan su estado actual y seleccionan automáticamente cursos, materiales y actividades específicas para optimizar su aprendizaje (Wang et al., 2025).

Gracias a este enfoque, el estudiante puede corregir deficiencias concretas y adquirir habilidades clave de manera más rápida y eficiente. Además, la personalización del itinerario educativo mejora considerablemente la motivación y el rendimiento académico.

Los ITS modernos han evolucionado hacia modelos basados en datos capaces de optimizar múltiples áreas críticas del aprendizaje. Entre ellas destacan la gestión del material educativo, el modelado dinámico del perfil del estudiante y la generación de retroalimentación adaptativa en tiempo real (Wang et al., 2025).

Comprender el verdadero estado del estudiante constituye uno de los elementos centrales de cualquier sistema de tutoría inteligente. La precisión en el diagnóstico del nivel de conocimiento y comportamiento del usuario determina la efectividad de la personalización y de la retroalimentación proporcionada por el sistema (Wang et al., 2025).

A diferencia de los enfoques tradicionales basados únicamente en modelos estáticos de aprendizaje automático, los ITS actuales utilizan LLM de forma proactiva para construir perfiles dinámicos y continuamente actualizados de cada estudiante (Wang et al., 2025).

Estos perfiles integrales se construyen a partir de diferentes dimensiones relacionadas con el comportamiento y rendimiento del usuario. Una de ellas corresponde a la adquisición de habilidades y al progreso cognitivo, permitiendo identificar las competencias ya adquiridas y las brechas de aprendizaje pendientes.

Otra dimensión importante corresponde a las preferencias de aprendizaje y asimilación del estudiante. Los ITS analizan qué formatos, actividades y recursos generan mejores resultados para cada usuario, adaptando dinámicamente los materiales educativos según sus preferencias individuales (Wang et al., 2025).

Asimismo, los sistemas inteligentes también analizan tendencias de comportamiento y participación, como la frecuencia de acceso, la duración de las sesiones o posibles patrones de desconexión. Gracias a ello, los ITS pueden detectar señales tempranas de fatiga o desmotivación y realizar intervenciones preventivas para mantener el compromiso del estudiante.

La expansión del aprendizaje electrónico (*e-learning*) ha incrementado significativamente la necesidad de sistemas capaces de ofrecer experiencias educativas más personalizadas y eficientes. Los estudiantes valoran especialmente este tipo de formación debido al ahorro de tiempo, dinero y recursos que proporciona (Shashiprabha et al., 2020).

Sin embargo, el crecimiento de las plataformas de aprendizaje online también ha incrementado la competencia dentro del sector educativo digital. Como consecuencia, las instituciones y proveedores de plataformas deben comprender mejor las percepciones, necesidades y expectativas de los estudiantes para diferenciarse y optimizar sus servicios (Shashiprabha et al., 2020).

La recopilación y análisis de datos relacionados con la experiencia del usuario se ha convertido en un componente esencial dentro de los sistemas inteligentes de aprendizaje. Para ello, se utilizan tanto datos cuantitativos como datos cualitativos obtenidos a partir del comportamiento y opiniones de los estudiantes.

En este contexto, el análisis de sentimientos constituye una herramienta especialmente relevante. Esta técnica de procesamiento del lenguaje natural permite identificar emociones, opiniones y percepciones expresadas en textos escritos por los usuarios (Shashiprabha et al., 2020).

Los algoritmos de clasificación de sentimientos, como Naive Bayes o Support Vector Machines (SVM), permiten analizar comentarios y reseñas para determinar si expresan satisfacción, crítica o neutralidad. No obstante, muchos modelos tradicionales presentan limitaciones al simplificar las opiniones únicamente en polaridades positivas o negativas (Shashiprabha et al., 2020).

La incorporación de análisis de sentimientos más avanzados permite comprender mejor la experiencia educativa del alumnado y adaptar los contenidos o metodologías según las necesidades detectadas. Esto favorece el desarrollo de plataformas educativas más eficientes y centradas en el usuario.

Paralelamente al desarrollo de los ITS, los Sistemas de Gestión del Aprendizaje (LMS) se han consolidado como la principal infraestructura tecnológica de la educación online. Estas plataformas integran la gestión de cursos, materiales, evaluaciones y herramientas de comunicación entre estudiantes y docentes (Soko et al., 2024).

Las funciones principales de los LMS incluyen la planificación académica, la administración de contenidos, el seguimiento del progreso del alumnado y la comunicación en tiempo real (Zamfiroiu y Vasile, 2024). Gracias a ello, estas plataformas permiten centralizar gran parte de la actividad educativa dentro de entornos virtuales.

Las tecnologías de aprendizaje online están transformando profundamente el diseño y la implementación de los procesos educativos. Su enfoque innovador proporciona mayores niveles de personalización, interactividad y eficiencia en comparación con los modelos tradicionales de enseñanza (Zamfiroiu y Vasile, 2024).

Los LMS modernos también incorporan funciones colaborativas, sistemas de seguimiento académico y herramientas de personalización adaptadas a las nuevas necesidades educativas. No obstante, todavía existen barreras relacionadas con el acceso tecnológico, la motivación del alumnado y la adaptación metodológica del profesorado (Abaricia et al., 2023, citado en Zamfiroiu y Vasile, 2024).

Dentro de este escenario, la evaluación adquiere un papel especialmente relevante. La evaluación educativa puede definirse como el proceso mediante el cual se determina y cuantifica el nivel de aprendizaje alcanzado por un estudiante dentro de un área específica (Salazar, 2020).

Los métodos de evaluación se relacionan directamente con las estrategias pedagógicas utilizadas por el profesorado. Según Salazar (2020), estos métodos pueden clasificarse en informales, semiformales y formales, dependiendo del nivel de planificación, control y percepción de evaluación por parte del estudiante.

La evaluación virtual o *e-assessment* constituye un componente esencial dentro del *e-learning*. Estas herramientas permiten proporcionar orientación y seguimiento continuo al estudiante dentro de entornos digitales de aprendizaje (Thomas et al., 2004, citado en Salazar, 2020).

Aunque la automatización de la evaluación ofrece ventajas significativas como rapidez, accesibilidad y coherencia, su efectividad depende de una adecuada planificación pedagógica y del correcto diseño de las actividades evaluativas (Salazar, 2020).

Uno de los enfoques más relevantes dentro de este ámbito es la evaluación adaptativa. Este modelo busca personalizar las pruebas según el nivel cognitivo, comportamiento, preferencias y necesidades específicas del estudiante (Baneres et al., 2015, 2016, citados en Salazar, 2020).

La evaluación adaptativa puede centrarse tanto en la ruta de aprendizaje como en la personalización de las actividades evaluativas. Por un lado, algunos sistemas seleccionan dinámicamente las actividades según la experiencia previa del usuario; por otro, otros modelos adaptan directamente las preguntas y contenidos del examen según el rendimiento del estudiante (Salazar, 2020).

Este tipo de evaluación informatizada se adapta continuamente al usuario teniendo en cuenta factores como el nivel de dificultad de las preguntas respondidas anteriormente. Gracias a ello, el sistema puede ofrecer pruebas ajustadas a la capacidad real del estudiante (Wainer et al., 2003; López, Sanmartín y Méndez, 2014, citados en Salazar, 2020).

En el marco de la evaluación adaptativa informatizada, la efectividad del proceso no concluye con la determinación del nivel de dificultad de la siguiente pregunta, sino que depende de manera crítica de la naturaleza de la retroalimentación o *feedback* que recibe el estudiante tras cada interacción. La literatura psicopedagógica distingue entre el feedback puramente verificativo (que se limita a indicar si la respuesta es correcta o errónea) y el feedback formativo o explicativo. Este último no solo señala el error, sino que expone el

razonamiento subyacente, desglosa el malentendido conceptual y provee pistas orientativas que estimulan la autorregulación del aprendizaje sin desvelar de forma inmediata la solución definitiva (Contreras-Espinosa y Eguia, 2022).

La integración de modelos de lenguaje (LLM) dentro de este bucle de evaluación adaptativa permite que los sistemas de tutoría inteligente generen este feedback explicativo en tiempo real y de forma completamente personalizada. Al analizar la traza específica del error cometido por el alumno, el modelo generativo puede adaptar el tono, la profundidad de la explicación y los ejemplos utilizados a las necesidades particulares del perfil del usuario. De este modo, el tratamiento del error dentro del *e-assessment* deja de ser un mero indicador punitivo de calificación cuantitativa para transformarse en un mecanismo activo de andamiaje cognitivo y diagnóstico pedagógico (Conejo et al., 2021).

Para lograr esta personalización, los ITS utilizan perfiles de usuario contruidos a partir de información relacionada con el comportamiento, intereses, limitaciones y preferencias del estudiante. Estos perfiles permiten almacenar y analizar datos relevantes para optimizar la experiencia educativa (Salazar, 2020).

En este sentido, los sistemas de recomendación desempeñan un papel fundamental dentro de las plataformas educativas inteligentes. A diferencia de los motores de búsqueda tradicionales, estos sistemas analizan las interacciones y comportamiento del usuario para recomendar recursos adaptados a sus necesidades específicas (Klašnja-Milićević et al., 2017; Ricci, Rokach y Shapira, 2011, citados en Salazar, 2020).

La combinación de ITS, sistemas de recomendación, análisis de comportamiento y retroalimentación adaptativa está transformando progresivamente los modelos tradicionales de educación online. Estas tecnologías permiten construir experiencias educativas más flexibles, personalizadas y centradas en el estudiante.

En este contexto, surge la necesidad de desarrollar plataformas capaces de integrar inteligencia artificial, personalización del aprendizaje y sistemas avanzados de tutorización dentro de un entorno unificado. Bajo esta perspectiva aparece Ilerna Smart, una aplicación orientada a mejorar los procesos de seguimiento académico, retroalimentación personalizada y adaptación dinámica del aprendizaje mediante tecnologías de inteligencia artificial y sistemas de tutoría inteligente. La plataforma busca optimizar la experiencia educativa del alumnado mediante recomendaciones personalizadas, generación automática de recursos y

sistemas de evaluación adaptativa capaces de ajustarse continuamente al progreso y necesidades específicas de cada estudiante.

2.3 Arquitectura de Microservicios en Plataformas Educativas

La arquitectura de microservicios representa uno de los cambios más significativos dentro del diseño moderno de software. Este modelo arquitectónico surge como respuesta a las limitaciones de las aplicaciones monolíticas tradicionales, proponiendo un enfoque basado en componentes independientes capaces de comunicarse entre sí mediante protocolos ligeros y estandarizados.

El término *microservicio* tuvo su origen durante un taller para arquitectos de software celebrado cerca de Venecia en el año 2011. Según Martin Fowler, reconocido ingeniero y especialista en diseño de software empresarial, este concepto comenzó a utilizarse para describir un nuevo estilo arquitectónico que diferentes desarrolladores estaban explorando simultáneamente de manera independiente (Barrios, 2018).

Aunque el término adquirió gran popularidad especialmente a partir de 2014, su idea fundamental no era completamente nueva. De hecho, muchos de los principios de los microservicios se encuentran estrechamente relacionados con la Arquitectura Orientada a Servicios (SOA), lo que ha generado dificultades para diferenciar claramente ambos conceptos dentro del ámbito tecnológico (Barrios, 2018).

Para comprender correctamente la arquitectura de microservicios resulta necesario compararla con el modelo monolítico tradicional. En este último, toda la aplicación se construye como una única unidad indivisible donde la interfaz de usuario, la lógica de negocio y el sistema de gestión de datos permanecen altamente acoplados dentro de un mismo bloque de software (Barrios, 2018).

En las aplicaciones monolíticas empresariales, cualquier modificación o actualización requiere intervenir sobre toda la estructura del sistema. Esto provoca que pequeñas modificaciones puedan afectar al funcionamiento completo de la plataforma, aumentando significativamente la complejidad del mantenimiento y el riesgo de errores críticos.

A medida que una aplicación monolítica crece y se incorporan nuevas funcionalidades, el volumen de código fuente aumenta considerablemente. Con el tiempo,

esta acumulación se convierte en un problema técnico importante, ya que localizar el punto exacto donde realizar modificaciones resulta cada vez más complejo y costoso (Barrios, 2018).

Además, debido al fuerte acoplamiento entre componentes, cualquier cambio requiere pruebas exhaustivas de regresión e implementaciones completas de la aplicación. Esta situación ralentiza los ciclos de desarrollo y despliegue, incrementando también los riesgos de seguridad y fallos operativos (Granados y Meza, 2024).

Como alternativa a estas limitaciones surge la arquitectura de microservicios, la cual propone construir aplicaciones como un conjunto de servicios independientes y modulares. Cada microservicio se ejecuta en su propio proceso y se comunica con el resto mediante mecanismos ligeros como APIs HTTP o sistemas de mensajería (Barrios, 2018).

La independencia entre microservicios permite aislar funcionalidades concretas dentro de componentes autónomos. Gracias a ello, los equipos de desarrollo pueden actualizar, escalar o desplegar servicios específicos sin afectar al resto de la plataforma.

Uno de los principales beneficios de este enfoque es la denominada escalabilidad quirúrgica. Esto significa que únicamente los servicios con mayor demanda consumen recursos adicionales de infraestructura, optimizando considerablemente el rendimiento y los costes operativos del sistema (Barrios, 2018).

La adherencia a la escalabilidad quirúrgica adquiere una dimensión crítica en las plataformas de capacitación durante los períodos de evaluación masiva cuando García-Valls et al. Pruebas de carga y estrés. (2022) muestran que, si bien las arquitecturas monolíticas sufren una degradación exponencial más allá de los tiempos de respuesta de 4000 ms y errores de tiempo de espera de la puerta de enlace para 500 usuarios simultáneos, los entornos basados en microservicios y Spring Boot mantienen latencias lineales estables por debajo de los 600 ms. Esta notable eficiencia se debe a la capacidad del sistema distribuido para replicar automáticamente horizontalmente solo los contenedores específicos que gestionan la lógica de prueba o las solicitudes API críticas, optimizando el uso de recursos y evitando con éxito el colapso de las funciones básicas del ecosistema tecnológico.

Asimismo, la arquitectura de microservicios mejora la tolerancia a fallos. Si un servicio presenta errores o problemas técnicos, estos quedan contenidos dentro de dicho componente sin comprometer necesariamente el funcionamiento global de la aplicación.

Este aislamiento funcional no sólo protege la disponibilidad del sistema, sino que también optimiza drásticamente el consumo de recursos de hardware como CPU y RAM, evitando que tareas pesadas ralenticen toda la aplicación. Aunque Rodríguez y Villalobos (2023) demostraron que los procesos de alta carga, como la serialización de grandes cantidades de datos JSON o el consumo de servicios externos de IA, pueden saturar el hilo de ejecución de un componente monolítico en un 100%, afectando a toda la plataforma, la arquitectura de microservicios limita este contenedor de carga algorítmica al contenedor de IA específico. De esta forma, si bien el microservicio mencionado utiliza los recursos necesarios para procesar solicitudes complejas, las funciones críticas de autenticación y persistencia académica funcionan con un nivel mínimo de CPU inferior al 12%, garantizando una navegación fluida y continua para el resto de estudiantes de la plataforma educativa.

Otra ventaja importante es la libertad tecnológica que proporciona este modelo arquitectónico. Cada microservicio puede desarrollarse utilizando diferentes lenguajes de programación, bases de datos o bibliotecas específicas según las necesidades concretas de cada funcionalidad (Barrios, 2018).

A diferencia de las aplicaciones monolíticas, donde todos los componentes comparten las mismas tecnologías y versiones, los microservicios permiten realizar actualizaciones y cambios tecnológicos de forma independiente sin desestabilizar el ecosistema completo.

Esta flexibilidad facilita enormemente la evolución tecnológica de las plataformas digitales y permite que distintos equipos trabajen simultáneamente sobre diferentes partes del sistema de forma autónoma y desacoplada.

No obstante, la implementación de microservicios también plantea desafíos relacionados con el diseño y tamaño de cada componente. Una de las preguntas más relevantes dentro de este modelo es determinar qué tan pequeño debe ser un microservicio (Barrios, 2018).

El tamaño ideal no se mide únicamente por el número de líneas de código, sino por su capacidad de ser gestionado eficientemente por un equipo reducido. Cuando un servicio se

vuelve demasiado complejo o difícil de mantener, se considera un indicador claro de que debe dividirse en componentes más pequeños (Barrios, 2018).

Las características principales de los microservicios se centran en tres aspectos fundamentales: su disponibilidad mediante red, su especialización funcional y su independencia operativa (Martínez, 2025).

Cada servicio se orienta a cumplir un único objetivo de negocio bien definido, permitiendo optimizar el software específicamente para esa funcionalidad concreta. Esta coherencia funcional favorece tanto el mantenimiento como la escalabilidad del sistema.

La comunicación entre microservicios se realiza exclusivamente mediante llamadas de red ligeras, como solicitudes HTTP o sistemas de mensajería asíncrona. Este modelo garantiza un bajo nivel de acoplamiento entre componentes y protege la independencia de cada servicio (Barrios, 2018).

La arquitectura de microservicios representa una evolución directa de los principios de la Arquitectura Orientada a Servicios (SOA). Este modelo estructural utiliza servicios reutilizables e interoperables como mecanismo central para aumentar la eficiencia, flexibilidad y productividad organizacional (De la Cruz, Espinosa y Cuba, 2019).

SOA permite integrar diferentes tecnologías, aplicaciones y sistemas heredados mediante APIs y mecanismos estandarizados de comunicación. Gracias a ello, las organizaciones pueden transformar infraestructuras rígidas en ecosistemas tecnológicos más ágiles y adaptables.

Según el modelo clásico de Krafzig, Banke y Slama (2004), SOA se estructura a partir de varios componentes principales. Entre ellos destacan las aplicaciones frontend, los servicios autónomos, los contratos de servicio, las implementaciones técnicas y los repositorios centralizados de descubrimiento de servicios (De la Cruz, Espinosa y Cuba, 2019).

Dentro de esta arquitectura, los contratos definen las reglas, restricciones y objetivos funcionales de cada servicio, mientras que las implementaciones contienen la lógica de negocio, bases de datos y configuraciones necesarias para su funcionamiento.

Los microservicios heredan gran parte de estos principios, aunque introducen un mayor nivel de independencia, modularidad y descentralización operativa respecto a los modelos SOA tradicionales.

En el ámbito educativo, las arquitecturas basadas en microservicios resultan especialmente útiles para desarrollar plataformas complejas y escalables capaces de gestionar múltiples funcionalidades simultáneamente.

Las plataformas educativas modernas deben integrar servicios relacionados con autenticación, gestión académica, generación de contenidos, inteligencia artificial, sistemas de evaluación, mensajería y análisis de datos. El enfoque monolítico dificulta considerablemente la evolución y mantenimiento de este tipo de ecosistemas tecnológicos.

Gracias a los microservicios, cada funcionalidad puede desarrollarse, desplegarse y mantenerse de manera independiente, facilitando tanto la evolución de la plataforma como la incorporación de nuevas tecnologías educativas.

La arquitectura *microfrontend* representa una extensión de estos principios hacia el desarrollo de interfaces de usuario. Este enfoque divide aplicaciones web complejas en módulos visuales autónomos e independientes gestionados por diferentes equipos de desarrollo (Granados y Meza, 2024).

Los *microfrontends* permiten implementar nuevas funcionalidades visuales sin afectar al resto de la aplicación, favoreciendo una mayor flexibilidad organizativa y acelerando los ciclos de desarrollo.

Además, este enfoque optimiza el rendimiento mediante técnicas de carga diferida (*lazy loading*), permitiendo que el navegador descargue únicamente los recursos necesarios según la interacción del usuario (Granados y Meza, 2024).

Dentro del ecosistema tecnológico actual, Spring Boot se ha consolidado como una de las herramientas más utilizadas para el desarrollo de arquitecturas de microservicios en Java (Rosado et al., 2023).

Spring Boot simplifica enormemente la configuración inicial de aplicaciones distribuidas gracias a su sistema de configuración automática y servidores integrados. Esto

permite construir componentes autónomos listos para producción con mayor rapidez y menor complejidad técnica.

Asimismo, Spring Boot se integra de forma nativa con Spring Cloud, facilitando la implementación de características fundamentales en sistemas distribuidos como descubrimiento de servicios, balanceo de carga, enrutamiento dinámico y tolerancia a fallos (Rosado et al., 2023).

En el desarrollo frontend, React se ha convertido en una de las tecnologías más utilizadas debido a su flexibilidad y enfoque basado en componentes reutilizables (Rosado et al., 2023).

La arquitectura modular de React resulta especialmente adecuada para estrategias de *microfrontend*, ya que permite que múltiples equipos trabajen sobre módulos visuales independientes sin interferencias entre sí.

En aplicaciones modernas, tanto frontend como backend suelen estructurarse mediante arquitecturas por capas. Este modelo divide el sistema en diferentes niveles funcionales claramente definidos.

Las entidades representan modelos de datos relacionados directamente con la base de datos. Por encima de ellas, los repositorios actúan como una capa de abstracción encargada de gestionar consultas y acceso a la información (Rosado et al., 2023).

La lógica de negocio se concentra dentro de la capa de servicios, mientras que los controladores gestionan las solicitudes del cliente y devuelven respuestas adaptadas a los protocolos de comunicación utilizados.

El pilar principal de esta estratificación del ecosistema distribuido es el modelo de diseño de bases de datos de microservicios, que resuelve una de las mayores debilidades de los LMS monolíticos: la saturación del grupo de conexiones de bases de datos centralizadas causada por solicitudes de escritura concurrentes. Según Pérez y Gómez (2021), al implementar microservicios independientes con Spring Boot, cada componente administra su propio almacén de datos de manera totalmente desacoplada; Esto garantiza que el escalado de escritura intensiva del módulo de evaluación de la plataforma no bloquee ni interrumpa las operaciones de lectura en el perfil o la base de datos de administración de contenido, eliminando por completo el riesgo de interbloqueos en el sistema de almacenamiento general.

La integración entre frontend y microservicios se realiza habitualmente mediante protocolos HTTP para operaciones síncronas y tecnologías de mensajería en tiempo real como STOMP para comunicaciones asíncronas (Rosado et al., 2023).

Gracias a este enfoque híbrido, las aplicaciones educativas modernas pueden proporcionar experiencias altamente interactivas, sincronización en tiempo real y una comunicación eficiente entre usuario y sistema.

La incorporación de arquitecturas de microservicios en plataformas educativas también facilita el desarrollo de sistemas inteligentes basados en inteligencia artificial, análisis de datos y aprendizaje adaptativo.

Servicios independientes dedicados al procesamiento de IA, generación de recomendaciones o análisis de comportamiento pueden desplegarse y escalarse de forma autónoma según las necesidades de la plataforma.

Además, esta arquitectura favorece la integración continua y el despliegue continuo (*CI/CD*), permitiendo implementar mejoras y nuevas funcionalidades de manera rápida y segura sin interrumpir el funcionamiento global del sistema.

Desde una perspectiva organizacional, los microservicios también facilitan la coordinación entre equipos de desarrollo pequeños y especializados, mejorando la productividad y reduciendo la complejidad operativa (Martínez, 2025).

En este contexto, la arquitectura de microservicios se presenta como una solución especialmente adecuada para plataformas educativas modernas que requieren alta escalabilidad, modularidad y capacidad de adaptación tecnológica.

Bajo esta perspectiva, Ilerna Smart adopta una arquitectura basada en microservicios con el objetivo de construir una plataforma educativa flexible, escalable y preparada para integrar funcionalidades avanzadas de inteligencia artificial y aprendizaje adaptativo. Gracias a este enfoque, la aplicación puede gestionar de forma independiente módulos relacionados con generación de contenido, sistemas de evaluación, tutorización inteligente, procesamiento de IA y analítica educativa, garantizando una evolución tecnológica más eficiente y una mejor experiencia para estudiantes y docentes dentro de entornos de formación online.

2.4 Arquitectura de Microservicios en Plataformas Educativas

La evolución de las plataformas educativas modernas está estrechamente relacionada con el desarrollo de ecosistemas tecnológicos robustos capaces de soportar aplicaciones distribuidas, entornos de ejecución multiplataforma y sistemas avanzados de persistencia de datos. En este contexto, lenguajes como Java y Kotlin, junto con frameworks empresariales como Spring Boot y herramientas de desarrollo especializadas, constituyen la base técnica de numerosas arquitecturas educativas contemporáneas orientadas al aprendizaje digital y los sistemas inteligentes (Deitel y Deitel, 2008).

La historia de Java se encuentra profundamente vinculada al auge de los microprocesadores y la expansión masiva de las computadoras personales durante finales del siglo XX. En la década de 1990, Sun Microsystems impulsó el denominado *Proyecto Green*, liderado por James Gosling, con el objetivo inicial de desarrollar un lenguaje para dispositivos electrónicos inteligentes. Aunque el proyecto enfrentó dificultades comerciales en sus primeras etapas, la aparición de la World Wide Web transformó completamente el destino de la tecnología, convirtiendo a Java en uno de los pilares fundamentales del software distribuido y de Internet (Deitel y Deitel, 2008).

Originalmente denominado *Oak*, el lenguaje recibió posteriormente el nombre de Java durante una reunión informal del equipo de desarrollo en una cafetería. Este cambio marcó el inicio de una tecnología que posteriormente se consolidaría como uno de los lenguajes más influyentes de la industria informática mundial, especialmente en sistemas empresariales, aplicaciones web y arquitecturas de microservicios (Deitel y Deitel, 2008).

La consolidación de Java estuvo impulsada principalmente por su capacidad multiplataforma, resumida en el principio *“Write Once, Run Anywhere”* (“Escribe una vez, ejecuta en cualquier lugar”). Esta característica permitió que los programas pudieran ejecutarse en distintos sistemas operativos sin necesidad de recompilar el código, facilitando enormemente el desarrollo de software distribuido y reduciendo la dependencia del hardware específico (Deitel y Deitel, 2008).

El fundamento técnico de esta portabilidad se encuentra en el uso del denominado *bytecode*, un formato intermedio generado por el compilador Java. A diferencia del código máquina tradicional, el *bytecode* no depende directamente de la arquitectura física del sistema, sino que es interpretado por la Máquina Virtual Java (JVM), permitiendo que el

misimo programa funcione de forma idéntica en diferentes entornos computacionales (Deitel y Deitel, 2008).

La Máquina Virtual Java (JVM) constituye el núcleo arquitectónico del ecosistema Java. Su función principal consiste en interpretar y ejecutar el *bytecode*, aislando al software de las particularidades del sistema operativo y del hardware subyacente. Gracias a esta abstracción, Java logró consolidarse como una plataforma altamente portable y adecuada para aplicaciones empresariales complejas y sistemas distribuidos (Deitel y Deitel, 2008).

Además de garantizar la portabilidad, la JVM incorpora mecanismos avanzados de optimización mediante compilación *Just-In-Time* (JIT). Este sistema analiza el comportamiento del programa en tiempo real e identifica las secciones de código ejecutadas con mayor frecuencia para traducirlas directamente a lenguaje máquina nativo, aumentando considerablemente el rendimiento de las aplicaciones (Deitel y Deitel, 2008).

Otro aspecto fundamental de Java es su enfoque orientado a objetos. A diferencia de los lenguajes procedimentales tradicionales, Java estructura el software mediante clases y objetos que encapsulan datos y comportamientos específicos. Este paradigma favorece la reutilización de código, la modularidad y el mantenimiento de sistemas complejos, aspectos esenciales en plataformas educativas modernas y arquitecturas de microservicios (Deitel y Deitel, 2008).

Las clases Java contienen métodos y atributos que permiten modelar entidades y operaciones del sistema. Esta estructura facilita la construcción de aplicaciones escalables y organizadas, donde cada componente cumple funciones específicas dentro de la lógica de negocio de la plataforma educativa.

La orientación a objetos también se apoya en principios fundamentales como encapsulación, herencia y polimorfismo. Estos mecanismos permiten desarrollar sistemas flexibles y extensibles, capaces de evolucionar con el tiempo sin comprometer la estabilidad global de la aplicación.

La seguridad constituye otra de las características más importantes del ecosistema Java. Durante la ejecución, la JVM incorpora un proceso de verificación de *bytecode* encargado de examinar las instrucciones del programa y garantizar que cumplan estrictamente con las reglas del lenguaje y las restricciones de seguridad (Deitel y Deitel, 2008).

Complementariamente, Java introdujo históricamente el modelo de seguridad conocido como *sandbox*, que permitía ejecutar código descargado desde Internet dentro de un entorno aislado y controlado. Este mecanismo resultó especialmente relevante en aplicaciones web distribuidas y sistemas conectados a redes externas.

La robustez del lenguaje también se refleja en su sistema de manejo estructurado de excepciones. Mediante bloques **try**, **catch** y **finally**, Java permite gestionar errores de ejecución de manera controlada, evitando fallos críticos que comprometan la estabilidad completa de la aplicación (Deitel y Deitel, 2008).

A esto se suma el mecanismo de recolección automática de basura (*garbage collection*), encargado de liberar memoria de objetos no utilizados. Esta característica reduce significativamente errores comunes asociados con la administración manual de memoria presentes en otros lenguajes de programación.

La programación concurrente representa otra ventaja importante del ecosistema Java. El soporte nativo para múltiples hilos de ejecución permite desarrollar aplicaciones capaces de procesar varias tareas simultáneamente, optimizando el uso de recursos y mejorando la capacidad de respuesta de plataformas educativas en tiempo real.

En sistemas educativos modernos, esta capacidad multiproceso resulta esencial para gestionar múltiples usuarios concurrentes, sesiones activas, servicios de evaluación automática y sistemas de retroalimentación instantánea sin afectar el rendimiento general de la plataforma.

El entorno de ejecución Java se complementa con el Java Runtime Environment (JRE), que proporciona los componentes mínimos necesarios para ejecutar aplicaciones Java, incluyendo la JVM y las bibliotecas principales del sistema (Deitel y Deitel, 2008).

Por otro lado, el Java Development Kit (JDK) constituye el conjunto completo de herramientas necesarias para desarrollar software Java. Este paquete incluye compiladores, bibliotecas, depuradores y utilidades de desarrollo que facilitan la construcción de aplicaciones empresariales complejas.

Dentro de este ecosistema, las API de Java juegan un papel fundamental. Estas bibliotecas proporcionan clases y componentes reutilizables para realizar operaciones

comunes, reduciendo considerablemente el tiempo de desarrollo y aumentando la confiabilidad del software.

La evolución de Java dio lugar a distintas ediciones especializadas según el contexto de aplicación. Java SE se orienta a aplicaciones generales; Java EE al desarrollo empresarial distribuido; y Java ME a dispositivos con recursos limitados como teléfonos móviles y sistemas embebidos (Deitel y Deitel, 2008).

En el ámbito actual del desarrollo móvil, Kotlin ha emergido como una alternativa moderna y eficiente para el ecosistema Android. Impulsado oficialmente por Google bajo la estrategia *Kotlin-first*, este lenguaje se ha convertido en el estándar principal para aplicaciones móviles nativas (Corilla, 2022).

Kotlin fue desarrollado por JetBrains en 2010 bajo la dirección de Andrei Breslav y posteriormente liberado como software de código abierto en 2012. Su diseño buscó ofrecer una sintaxis más concisa y segura que Java, manteniendo al mismo tiempo compatibilidad total con la JVM y el ecosistema existente (Corrilla, 2022).

Una de las principales ventajas de Kotlin es la reducción significativa de código repetitivo, mejorando la legibilidad y disminuyendo errores comunes de programación. Estas propiedades han favorecido su rápida adopción en aplicaciones Android y sistemas empresariales modernos.

Kotlin también destaca por su capacidad multiplataforma mediante Kotlin Multiplatform (KMP), permitiendo compartir lógica de negocio entre aplicaciones Android, iOS y sistemas backend, optimizando tiempos de desarrollo y mantenimiento.

En el contexto educativo, el aprendizaje móvil (*M-Learning*) ha adquirido gran relevancia gracias a la expansión de dispositivos móviles y aplicaciones educativas inteligentes. Estas tecnologías permiten acceder al contenido educativo desde cualquier lugar y momento, promoviendo modelos de aprendizaje ubicuos y flexibles (Corilla, 2022).

El ecosistema Android constituye actualmente uno de los principales entornos para el desarrollo de aplicaciones educativas móviles. Basado en el kernel Linux y diseñado específicamente para dispositivos táctiles, Android proporciona una arquitectura flexible y altamente extensible (Fragoso, 2022).

Para el desarrollo de aplicaciones Android, Android Studio se consolidó como el entorno de desarrollo integrado (IDE) oficial impulsado por Google. Basado en IntelliJ IDEA, este IDE integra herramientas avanzadas de programación, diseño visual, emulación y depuración (Bermúdez, 2021).

Android Studio optimiza el ciclo completo de desarrollo móvil mediante funciones como análisis de rendimiento, emulación avanzada, integración con Gradle y soporte nativo para Kotlin y Java.

El sistema de construcción basado en Gradle permite automatizar compilaciones, gestionar múltiples configuraciones y administrar dependencias de manera eficiente. Esta capacidad resulta esencial para proyectos educativos complejos con múltiples módulos y servicios integrados.

Además, Android Studio incorpora herramientas avanzadas de análisis estático como *Lint*, encargadas de detectar errores de seguridad, rendimiento y accesibilidad antes de la ejecución de la aplicación.

El ecosistema de desarrollo moderno también depende fuertemente de herramientas de control de versiones. En este ámbito, Git y GitHub se consolidaron como estándares fundamentales para gestionar cambios, colaborar entre equipos y mantener trazabilidad del software.

Android Studio integra directamente estos sistemas de control de versiones, permitiendo ejecutar operaciones como *commit*, *push*, *pull* y resolución de conflictos mediante interfaces visuales intuitivas (Bermúdez, 2021).

En paralelo al desarrollo móvil, las aplicaciones web empresariales han evolucionado gracias a frameworks robustos como Spring Framework y Spring Boot. Estas tecnologías permiten construir sistemas escalables, seguros y altamente mantenibles.

Spring Boot surgió como una extensión de Spring Framework basada en el principio de “*Convención sobre configuración*”, eliminando configuraciones complejas y automatizando gran parte de la infraestructura necesaria para ejecutar aplicaciones empresariales (Ramírez, 2022).

La principal ventaja de Spring Boot es su capacidad de configuración automática (*Auto-Configuration*), que analiza las dependencias del proyecto y genera automáticamente los componentes necesarios para el funcionamiento del sistema.

Este enfoque permite que los desarrolladores se concentren principalmente en la lógica de negocio, reduciendo significativamente el tiempo de implementación y minimizando errores derivados de configuraciones manuales complejas.

Dentro de Spring Boot destacan los denominados *starters*, paquetes de dependencias preconfiguradas que simplifican enormemente la integración de funcionalidades específicas como desarrollo web, persistencia de datos o seguridad.

Por ejemplo, el *starter* `spring-boot-starter-web` facilita la creación de servicios REST, mientras que `spring-boot-starter-data-jpa` simplifica la integración con bases de datos relacionales mediante tecnologías ORM.

La persistencia de datos constituye un componente crítico en plataformas educativas modernas. Los sistemas LMS y aplicaciones inteligentes requieren almacenar grandes volúmenes de información relacionados con usuarios, progreso académico, evaluaciones y comportamiento de aprendizaje.

En este contexto, frameworks como Spring Data JPA permiten abstraer la complejidad del acceso a bases de datos, facilitando operaciones CRUD y consultas avanzadas mediante repositorios y entidades.

La arquitectura de capas utilizada en aplicaciones empresariales modernas divide el sistema en componentes especializados. Las entidades representan modelos de datos; los repositorios gestionan la persistencia; los servicios encapsulan la lógica de negocio; y los controladores procesan solicitudes externas.

Esta separación estructural favorece la mantenibilidad, reutilización y escalabilidad del sistema, aspectos fundamentales en plataformas educativas distribuidas y arquitecturas basadas en microservicios.

La gestión de dependencias y automatización de proyectos se apoya frecuentemente en Apache Maven, una herramienta que centraliza compilación, empaquetado, pruebas e integración continua mediante archivos POM configurables (Ramírez, 2022).

Maven permite integrar bibliotecas externas desde repositorios centralizados y automatizar procesos complejos del ciclo de vida del software, optimizando la productividad de los equipos de desarrollo.

Para acelerar aún más el inicio de proyectos empresariales, Spring Initializr proporciona una plataforma web capaz de generar automáticamente la estructura base de aplicaciones Spring Boot listas para importar en cualquier IDE.

Los entornos de desarrollo modernos también incluyen herramientas ligeras y altamente extensibles como Visual Studio Code. Este editor multiplataforma destaca por su rapidez, soporte para múltiples lenguajes y amplio ecosistema de extensiones (Code, 2019).

VS Code integra funcionalidades avanzadas como depuración, control de versiones, análisis estático y autocompletado inteligente mediante IntelliSense, mejorando considerablemente la productividad del desarrollador.

Su naturaleza multiplataforma facilita además la colaboración entre equipos distribuidos que trabajan sobre distintos sistemas operativos como Windows, Linux o macOS.

En el ámbito educativo, estas herramientas tecnológicas permiten desarrollar plataformas escalables orientadas al aprendizaje digital, integrando aplicaciones móviles, servicios web y sistemas inteligentes de recomendación.

La combinación de Java, Kotlin, Spring Boot y Android Studio conforma un ecosistema tecnológico sólido para la implementación de sistemas educativos modernos basados en microservicios y aprendizaje personalizado.

Asimismo, la integración de herramientas de persistencia de datos, automatización de proyectos y control de versiones garantiza la estabilidad, mantenibilidad y evolución continua de las plataformas educativas inteligentes.

Finalmente, el ecosistema de desarrollo contemporáneo demuestra que la construcción de entornos educativos digitales no depende únicamente de interfaces visuales o contenidos pedagógicos, sino también de arquitecturas técnicas robustas capaces de soportar escalabilidad, seguridad, interoperabilidad y procesamiento inteligente de datos en tiempo real.

3.0 OBJETIVOS

3.1 Objetivo principal

El objetivo principal de este proyecto es el diseño, desarrollo y despliegue de un e-learning que resuelva las carencias educativas de la formación a distancia o online. Para ello, se idea la creación de una solución de software avanzada que será usada como apoyo para el alumnos con el fin de fusionar las capacidades del procesamiento del lenguaje natural (“Para que los propios actores del flujo de trabajo pueda interactuar con la IA”) con arquitecturas distribuidas de alta disponibilidad (“Arquitectura de micro-servicios que permite una alta escalabilidad”).

3.2 Subobjetivos específicos

Con el fin de poder cumplir con el objetivo principal se debe de desarrollar subobjetivos internos del proyecto, estos subobjetivos están distribuidos de manera estratégica a lo largo de las capas del software

1. Capa de Inteligencia Artificial: Automatización Evaluativa y Consistencia Semántica

El primer subobjetivo del proyecto se orienta al diseño e implementación de una capa de inteligencia artificial con la API de Grok, que sea capaz de automatizar procesos evaluativos dentro de la plataforma educativa con el fin de poder mejorar el rendimiento académico de los alumnos. Este componente busca aprovechar las capacidades del modelo de lenguaje Grok a gran escala ya sea para generar cuestionarios académicos, resúmenes personalizados por cada documento, clasificar los errores de los cuestionarios y proporcionar retroalimentación personalizada al alumnado por cada error. El objetivo principal de este subobjetivo es aliviar la carga laboral del profesorado con el fin de poder atender a los alumnos y aumentar la rapidez con la que los estudiantes reciben feedback personalizados sobre su desempeño académico a través de una app.

Este subobjetivo se centra en la integración directa entre los micro-servicios de la aplicación y el modelo de inteligencia artificial Grok mediante el uso de la (API Key) privada, esta api es proporcionada por el servicio, ya que Grok permite una plan gratuito de 30 solicitudes por minuto. La comunicación se realizará a través de peticiones HTTP seguras y estructuras de intercambio de datos en formato JSON, desde dentro el propio backend envia

automaticamente “prompts” personalizados dependiendo del proceso, Grok captará la documentación en la base de datos, comenzará a analizar y se generará tests y resúmenes. De esta manera gracias a esta conexión directa mediante la API Key, la plataforma podrá consumir de forma eficiente las solicitudes (“Tokens”), servicios de generación y análisis de contenido ofrecidos por Grok, recibiendo respuestas estructuradas que posteriormente serán interpretadas, validadas y gestionadas por los distintos módulos del sistema.

El funcionamiento general de esta capa de inteligencia artificial se fundamenta en el proceso automatización de los análisis de los documentos. El profesorado podrá adjuntar formación académica en formato PDF oficial de las asignaturas, como apuntes, presentaciones, ejercicios o documentación técnica. Una vez cargados, el sistema deberá procesar dichos documentos y transformarlos en JSON para que el modelo de IA, pueda comprender el documento y generar evaluaciones ajustadas sobre la documentación adjuntada.

La ingeniería de prompts se estructurará mediante una separación clara entre el System Message (“Es el prompt donde se le informa a la propia IA como interactuar con el prompt que recibirá por parte del usuario además es fundamental para un agente de IA”) y el prompt por parte del usuario (“El proceso que deberá de realizar”). El System Message actuará interpretando unas instrucciones / órdenes donde se definirá el rol que el modelo debe tener para ejecutar el proceso, obligándolo a comportarse como un evaluador académico técnico y restringiendo cualquier exageración / alucinación que pueda llegar a tener. Por otro lado, el prompt por parte del usuario contendrá las instrucciones detalladas relacionadas con el proceso de generación de preguntas en forma de test y análisis de respuestas.

Estará limitada las alucinaciones restringiendo el nivel de libertad que pueda llegar a tener el modelo de IA, de esta manera solo ejecutará de forma precisa las instrucciones que tiene y no divagará o intentará crear nuevas formas de ejecutar las instrucciones, se fijará aproximadamente en 0.0 y 0.2.

La generación de retroalimentación personalizada es otro de los objetivos de este subobjetivo. Una vez completado, enviado y corregido, el modelo de IA analizará el cuestionario para comprender los errores cometidos por parte del estudiante analizando cada pregunta fallida y producirá resúmenes explicativos relacionados con los conceptos mal comprendidos o mal ejecutados por parte del alumno. Esto permitirá construir nuevas metodologías de aprendizaje más adaptativas y centradas para la formación individual de cada alumno.

Desde el punto de vista de la estabilidad del profesorado, este subobjetivo contribuye a reducir la sobrecarga administrativa del profesorado permitiendo así el poder delegar al sistema tareas repetitivas que antes administrada el profesorado como la elaboración de preguntas, clasificación de respuestas y generación de feedback explicativos.

El sistema de backend para tener un sistema de control de errores relacionados con la IA Grok, al ser usada en este momento de forma de prueba, es posible que produzca errores de entendimiento o exceso de cuota diaria, por tanto en su mismo proceso se ha aplicado timeouts donde se ha analizado el tiempo medio que pueda llegar a pensar para poder generar test, resúmenes o feedbacks personalizados que en el caso que supere por 3 -5 segundos dicha cantidad saltará un error advirtiéndole al usuario que en este momento no está operativo y se ejecutará por cada 2- 3 minutos un reinicio automático, con el fin de poder continuar con el proceso.

En conjunto, este subobjetivo busca construir una infraestructura inteligente capaz de combinar automatización, precisión semántica y personalización educativa. La integración de modelos de lenguaje avanzados no se plantea únicamente como una herramienta tecnológica, sino como un componente estratégico orientado a transformar los procesos de evaluación y retroalimentación dentro del ecosistema educativo digital.

2. Capa de Servidor y Arquitectura: Modularidad Distribución y Escalabilidad Horizontal

Este subobjetivo es para lograr implementar la arquitectura de micro-servicios en el desarrollo de la arquitectura backend de la plataforma, de esta manera se sustituye el modelo monolítico ya que para proyecto de gran magnitud puede ocasionar problemas económicos y de escalabilidad, por lo tanto se diseñó la arquitectura de servicios para mejorar la modularidad, la estabilidad y la capacidad del sistema. Para ello, se utilizará Java 17 JDK ya que es el más estable y la versión que más dependencias usables en la actualidad permite junto con Spring Boot como base tecnológica principal para la construcción de los servicios del servidor.

La plataforma estará dividida en diferentes microservicios independientes, encargados de funciones específicas como el servicio de autenticación, servicio de los documentos, servicio de los tests, servicio de las asignaturas y un api gateway que permite unir todos estos backends que permite que el frontend y los backend de microservicios se puedan comunicar. Esta separación permitirá que cada módulo funcione de manera autónoma sin afectar directamente al resto del sistema.

Como antes mencionado si ocurre algún fallo en algún servicio del backend este no afectará a todos por tanto cada microservicio tiene un control de errores donde se gestionará por alertas informando al usuario del estado del servicio. El microservicio encargado de procesar las respuestas generadas por la inteligencia artificial maneja operaciones más pesadas de forma independiente, evitando que estas afecten al rendimiento general de la plataforma, por eso se le permite acceder a la base de datos para poder usar información que ya analizó previamente para el uso eficiente de solicitudes.

Gracias a este y control de errores, funcionalidades críticas como el servicio de autenticación o el servicio de asignatura podrán mantenerse estables incluso durante momentos de alta concurrencia o procesamiento intensivo de datos provenientes de la IA.

Además, la arquitectura de microservicios permite una mayor escalabilidad horizontal, facilitando la incorporación de nuevos servicios con el fin de distribuir la carga entre los distintos servicios activos según las necesidades del sistema y el crecimiento de usuarios de la plataforma.

Finalmente, este enfoque distribuido mejorará el mantenimiento, la actualización y la evolución futura del software, permitiendo incorporar nuevas funcionalidades sin comprometer la estabilidad global del ecosistema educativo.

3. Capa de Cliente y Multiplataforma): Interactividad, Reactividad y Fluidéz Móvil

Este subobjetivo se centra en el desarrollo de la capa cliente de la aplicación, priorizando una experiencia de usuario fluida, rápida e interactiva tanto en web como en móvil si bien son diferentes herramientas para su desarrollo el objetivo es que además de cumplir con la fluidez, que tengan cohesión entre ellas de esta manera los estudiantes y profesores puedan acceder al sistema desde distintos dispositivos sin problemas de rendimiento ni limitaciones de compatibilidad.

Para la versión web de la plataforma se utilizará React con la plantilla de proyecto de Vite. Esta librería permite que el diseño web sea completamente dinámico y accesible permitiendo el uso de componentes, además al ser dinámica los cambios se producen sin necesidad de recargar completamente la página en cada interacción del usuario debido al DOM virtual que tiene React, mejorando considerablemente la velocidad de navegación y la experiencia visual.

React a usar una arquitectura basada en componentes facilita la reutilización de ellos para interfaz, permitiendo construir componentes independientes como paneles académicos, reportes de errores, evaluaciones o estadísticas de rendimiento.

Uno de los elementos más importantes de React que permite que sea tan dinámico es el uso del Virtual DOM de React, ya que la propia herramienta crea un árbol con el Virtual DOM que representa la interfaz de usuario actual. Luego, genera el DOM real a partir de este para que el usuario pueda verlo de esta manera el renderizado de la interfaz se actualiza únicamente los componentes que cambian dentro de la pantalla.

Además, se implementarán técnicas de carga diferida (*lazy loading*) para reducir el consumo innecesario de recursos. Esto permitirá que determinados módulos o componentes de la aplicación solo se carguen cuando el usuario realmente los necesite, mejorando el tiempo de respuesta general de la plataforma.

En el apartado móvil, la aplicación será desarrollada de forma nativa para Android utilizando Kotlin dentro del entorno Android Studio. La elección de Kotlin responde a su integración oficial con Android y a sus ventajas relacionadas con seguridad, rendimiento y programación moderna.

Uno de los principales desafíos técnicos en el desarrollo móvil será evitar el bloqueo del hilo principal de la interfaz de usuario (*UI Thread*) durante las comunicaciones con la API del backend. Si estas operaciones se ejecutarán directamente en el hilo principal, la aplicación podría sufrir congelaciones o retrasos visibles para el usuario y el diseño técnico ya que al usar como diseño XML cada layout es completamente independiente de éste por tanto se tiene que configurar el flujo entre layouts para que el usuario se sienta cómodo en la aplicación.

Para solucionar este problema se implementará programación asíncrona mediante corrutinas de Kotlin. Este mecanismo permitirá realizar peticiones de red, procesamiento de datos y comunicación con los microservicios en segundo plano sin interrumpir la interacción del estudiante con la aplicación.

La integración entre React en la versión web y Kotlin en la aplicación móvil permitirá construir un ecosistema multiplataforma coherente, donde ambos clientes compartan la misma infraestructura de microservicios y acceso a datos, garantizando consistencia funcional en todos los entornos.

Finalmente, este subobjetivo busca consolidar una capa cliente moderna, reactiva y optimizada, capaz de ofrecer accesibilidad, velocidad y estabilidad tanto en ordenadores como en dispositivos móviles dentro del ecosistema educativo propuesto.

4. Capa de Cliente y Multiplataforma): Interactividad, Reactividad y Fluidez Móvil

Este subobjetivo se centra en la gestión y persistencia de los datos sensibles de la plataforma educativa Ilerna Smart, incluyendo documentación académica, datos sensibles del alumnado y profesores, calificaciones y registros generados por la IA Grok. Para ello, se implementará una base de datos relacional utilizando MySQL como motor principal de almacenamiento.

El diseño de la base de datos buscará garantizar orden, seguridad y eficiencia en el manejo de la información, permitiendo que los datos puedan ser disponibles y estructurados de forma eficiente incluso ante grandes volúmenes de operaciones simultáneas.

El modelo relacional se diseñará aplicando principios de normalización de datos hasta alcanzar la Tercera Forma Normal (3FN). Este enfoque permite reducir la redundancia de información y evitar inconsistencias dentro del sistema de almacenamiento.

Asimismo, se utilizarán claves primarias (“Para la identificación de las entidades”)y claves foráneas (“Para la identificar las relaciones entre las entidades”) , estableciendo relaciones seguras entre las diferentes entidades compuestas por el Modelo Relacional de la aplicación de Ilerna Smart

El diseño de persistencia buscará prevenir problemas de saturación del pool de conexiones y bloqueos mutuos (*deadlocks*) durante periodos de alta concurrencia, especialmente en procesos relacionados con evaluaciones automáticas y almacenamiento masivo de respuestas.

Finalmente, este subobjetivo pretende consolidar una infraestructura de datos robusta, escalable y segura, capaz de soportar el crecimiento de la plataforma educativa y garantizar la integridad de toda la información académica procesada por el sistema.

4.0 METODOLOGÍA

El desarrollo de una software con arquitectura distribuida, multiplataforma e integrada con inteligencia artificial como Ilerna Smart requiere la implementación de una metodología flexible, y altamente adaptable para cambios funcionales y técnicos en el desarrollo de la aplicación, por eso se implementó una metodología ágil.

En este contexto, las metodologías tradicionales de desarrollo en cascada (*Waterfall*), son características por tener fases secuenciales rígidas y una validación tardía del software, para este tipo de aplicación no resultan adecuadas debido al ecosistema basado en microservicios, integración continua y consumo de APIs

Por tanto, se seleccionó una metodología basada en las Metodologías Ágiles como antes mencionada, concretamente en Scrum, complementado con prácticas inspiradas en DevOps para la gestión de despliegues, pruebas e implementación de los distintos microservicios que conforman la aplicación, esta metodología fue vista y desarrollada en años anteriores como una de las más usadas para empresas que desarrollan software .

Este enfoque permite segmentar el desarrollo del proyecto en ciclos iterativos (“Sprints”) reduciendo costos por errores de arquitectura, enfoque en objetivos por cada dos semanas con el fin de alcanzar el objetivo mensual y siendo más supervisado que proyectos con revisión única mensual

4. Marco de Trabajo Ágil: Scrum Adaptado

La metodología Scrum organiza el desarrollo del software mediante ciclos conocidos como **Sprints**, las cuales en este proyecto poseen una duración aproximada de entre 1 -2 semanas. Cada Sprint tiene como objetivo desarrollar un servicio del sistema, permitiendo comprobar el funcionamiento por cada servicio creado antes de avanzar hacia etapas posteriores del desarrollo de la aplicación.

Este modelo iterativo resulta especialmente adecuado para un proyecto como Ilerna Smart, ya que la inteligencia artificial, arquitecturas distribuidas y aplicaciones multiplataforma requiere constantes procesos de prueba, ajuste y refinamiento técnico sobre todo para conseguir lo que busca ya que con el uso de IA, aunque se use un modelo ya existente siempre tiene que adaptarse como se aplica sus conocimiento en la aplicación.

Dentro de la organización operativa del proyecto se definieron los siguientes elementos metodológicos:

Product Backlog

El Product Backlog representa el repositorio central de requisitos funcionales y técnicos del sistema. Todos los servicios de la aplicación antes de diseñarlos se realiza la comprobación de usabilidad/enfoque por tanto fueron descompuestos en resúmenes breves para entender que comportamiento tendría una vez implementada

Por ejemplo:

- “El alumno quiere realizar cuestionarios generados automáticamente por IA con la información recolectada del fichero adjunto del profesor, para evaluar sus conocimientos y así poder entender si conoce bien el tema o necesita un feedback personalizado respecto a sus preguntas fallidas”.
- “El profesor, quiere que la plataforma genere automáticamente cuestionarios mediante inteligencia artificial a partir de la documentación que adjunto, permitiendo evaluar de forma dinámica a los estudiantes y detecta errores frecuentes para ofrecer un seguimiento más preciso de ellos”.

Sprint Backlog

El Sprint Backlog corresponde al conjunto de tareas seleccionadas para desarrollarse durante cada Sprint. Estas tareas incluyen actividades de programación, diseño de arquitectura, integración de APIs, modelado de bases de datos, pruebas y validación de componentes.

Cada Sprint fue diseñado para construir funcionalidades completas y funcionales del ecosistema, evitando dependencias excesivas entre módulos.

Daily Scrum

Las reuniones de seguimiento diarias permiten evaluar el progreso técnico del proyecto, identificar bloqueos, reorganizar prioridades a corto plazo e implementar nuevas ideas de mejora respecto a lo actual.

Estas sesiones resultaron especialmente importantes para resolver incidencias relacionadas con:

- Integración de la API de Grok.
- Problemas de concurrencia entre microservicios.
- Consumo asíncrono de datos desde aplicaciones móviles Android.

En este caso como el proyecto lo hizo una sola persona, le pedía consejos a diferentes IAs (“Actuando como un jefe de proyecto de la aplicación”) con el fin de evaluar el progreso, las ideas y los objetivos implementados

4.2 Planificación de los Sprints

El desarrollo de Ilerna Smart se estructuró en cuatro grandes fases iterativas, cada una enfocada en un área específica de la arquitectura del sistema.

Sprint 1: Infraestructura y Persistencia de Datos

La primera fase se centró en la construcción de la infraestructura principal del backend y la configuración inicial del entorno distribuido.

Durante este Sprint se realizó el modelado de la base de datos relacional en MySQL aplicando principios de normalización hasta alcanzar la Tercera Forma Normal (3FN).

Además se analizó durante una semana entidades potenciales pendientes de agregar en el desarrollo de aplicación

Se realizaron numerosas consultas multitaslas con el fin de evaluar las conexiones entre las entidades de la aplicación, una vez se ha comprobado la infraestructura del software, se comienza a desarrollar los servicios.

Sprint 2: Desarrollo del Backend e Integración de Inteligencia Artificial

Una vez consolidada la infraestructura distribuida, el segundo Sprint se enfocó en el desarrollo de la lógica de negocio y la integración con la inteligencia artificial Grok.

Durante esta etapa se desarrollaron los microservicios funcionales del sistema mediante Java 17 JDK y Spring Boot, implementando módulos independientes para usuarios, evaluaciones y gestión académica.

Asimismo, se programó el consumo de la API de Grok mediante peticiones HTTP autenticadas con API Key, permitiendo el envío de prompts académicos y documentación docente.

También se diseñaron mecanismos de ingeniería de prompts para forzar respuestas estructuradas en formato JSON evitando alucinaciones y facilitando posteriormente su procesamiento e inserción automática en la base de datos.

Sprint 3: Desarrollo Multiplataforma (Web y Android)

Con el backend completamente operativo y exponiendo endpoints REST funcionales comprobados con la aplicación “Postman”, el tercer Sprint se centró en la construcción de las interfaces cliente.

Para la versión web se desarrolló una aplicación SPA utilizando React junto con Vite, permitiendo una experiencia dinámica basada en el Virtual DOM y renderizado eficiente de componentes.

El panel docente fue diseñado para visualizar estadísticas académicas por cada asignatura y resultados generados por la inteligencia artificial sin necesidad de recargar continuamente la interfaz.

De forma paralela, se desarrolló la aplicación móvil nativa Android utilizando Kotlin y Android Studio.

En esta fase se implementaron corrutinas de Kotlin para gestionar peticiones asíncronas hacia los microservicios sin bloquear el hilo principal de la interfaz (*UI Thread*), garantizando una navegación fluida y estable para el estudiante.

Sprint 4: Pruebas, Integración y Validación del Sistema

La última fase del proyecto estuvo orientada a la validación integral del ecosistema software antes de su cierre final.

Se realizaron pruebas unitarias e integraciones sobre controladores, servicios y componentes internos de Spring Boot para verificar la estabilidad funcional de los microservicios.

Además, se ejecutaron pruebas de carga y concurrencia con el objetivo de validar:

- El aislamiento del microservicio de inteligencia artificial.
- La estabilidad del Gateway ante múltiples peticiones simultáneas.
- El correcto comportamiento del pool de conexiones de MySQL.
- La persistencia segura de las calificaciones académicas.

Finalmente, se verificó que los tiempos de respuesta generales del sistema se mantuvieran estables incluso bajo escenarios de alta concurrencia, garantizando así la escalabilidad y resiliencia de la arquitectura distribuida implementada.

5.0 CONTRIBUCIÓN DEL TRABAJO

Se presenta la contribución técnica y funcional desarrollada en el proyecto Ilerna Smart, describiendo de forma detallada el diseño, implementación y validación del ecosistema software. La aportación principal del trabajo se centra en la integración de inteligencia artificial generativa dentro de una arquitectura microservicios, permitiendo automatizar procesos de evaluación académica y generación de contenido educativo personalizado.

Asimismo, se expone la estructura completa de la arquitectura implementada, incluyendo la comunicación entre microservicios, la integración de la API de inteligencia artificial Grok, el desarrollo multiplataforma de las interfaces cliente y la gestión de persistencia de datos mediante bases de datos relacionales. También se describen las tecnologías utilizadas durante el desarrollo, los patrones de diseño aplicados y las decisiones arquitectónicas adoptadas para garantizar escalabilidad, modularidad y rendimiento.

Finalmente, el capítulo incorpora un caso práctico de funcionamiento donde se valida el comportamiento real de la plataforma mediante la ejecución de los distintos módulos desarrollados, permitiendo comprobar la correcta generación automática de cuestionarios, el procesamiento de respuestas, la persistencia de resultados académicos y la interacción entre los diferentes componentes del sistema.

5.1 Arquitectura Técnica del Sistema

La comunicación entre servicios se realiza mediante protocolos HTTP y APIs REST, permitiendo el intercambio estructurado de información entre los distintos módulos del backend y las aplicaciones cliente. Gracias a esta distribución lógica, cada microservicio puede evolucionar, desplegarse y escalar de manera independiente según las necesidades del sistema.

Para coordinar el flujo de peticiones dentro de la arquitectura distribuida, el sistema implementa un API Gateway como punto centralizado de acceso. Bajo esta topología, las aplicaciones cliente desarrolladas en React y Kotlin no interactúan directamente con los microservicios internos, sino que todas las solicitudes son enviadas inicialmente a la pasarela perimetral. Este componente se encarga de inspeccionar las rutas de cada petición y redirigirlas automáticamente al microservicio correspondiente de forma transparente para el usuario.

Esta estrategia permite ocultar la estructura interna del backend, centralizar la seguridad del sistema y simplificar la gestión del tráfico de red. Además, evita exponer direcciones IP internas o puertos específicos de los servicios, incrementando la protección y estabilidad de la infraestructura.

Justificación de la Suite de Microservicios

Con el objetivo de cumplir el principio de responsabilidad única (*Single Responsibility Principle*), el dominio funcional de Ilerna Smart fue fragmentado en distintos microservicios especializados, cada uno encargado de una tarea concreta dentro del ecosistema académico.

API Gateway (api-gateway)

El API Gateway actúa como el único punto de entrada para las aplicaciones cliente. Su función principal consiste en enrutar las peticiones HTTP hacia los diferentes microservicios internos según la URL solicitada. Asimismo, centraliza políticas de seguridad y configuraciones de intercambio de recursos (*CORS*), funcionando como una capa de protección perimetral que evita accesos directos a los servicios internos del backend.

Microservicio de Autenticación y Usuarios (auth-service)

El microservicio de autenticación encapsula toda la lógica relacionada con el registro de usuarios, validación de credenciales y control de accesos. Este módulo implementa autenticación basada en tokens JWT (*JSON Web Tokens*), permitiendo mantener sesiones seguras y desacopladas.

Gracias a su independencia arquitectónica, posibles ataques de fuerza bruta o picos masivos de autenticación quedan confinados únicamente a este servicio, evitando afectar el rendimiento del resto de módulos académicos del sistema.

Microservicio de Asignaturas (subject-service)

El microservicio de asignaturas gestiona toda la estructura académica de la plataforma. Entre sus responsabilidades se encuentran la administración de módulos formativos, asignación de profesores, matriculación de alumnos y organización del contenido educativo.

Además, proporciona la información necesaria para construir dinámicamente los paneles personalizados de profesores y estudiantes dentro de las interfaces cliente.

Microservicio de Documentación (document-service)

Este microservicio administra el almacenamiento y procesamiento de los recursos didácticos proporcionados por el profesorado, especialmente los documentos PDF utilizados como fuente principal de información académica.

Su relevancia dentro de la arquitectura radica en que actúa como proveedor oficial de contenido para el sistema de inteligencia artificial, suministrando el material teórico que posteriormente será procesado para generar cuestionarios automáticos y análisis semánticos.

Microservicio de Tests e Inteligencia Artificial (test-service)

El microservicio de tests e inteligencia artificial constituye el núcleo computacional más importante de la plataforma y representa la principal contribución técnica del proyecto.

Este componente establece comunicación directa con la API de Grok mediante peticiones HTTP autenticadas utilizando API Key. Su flujo interno incluye la recepción de documentación académica desde el MS-Documentación, la construcción de prompts especializados, el envío de instrucciones al modelo de inteligencia artificial y el procesamiento automatizado de las respuestas generadas.

Asimismo, el servicio analiza las respuestas JSON devueltas por la IA, almacena automáticamente las preguntas generadas en la base de datos y ejecuta procesos de corrección automática de cuestionarios, proporcionando retroalimentación personalizada y reportes de errores en tiempo real para cada estudiante.

Finalmente, el aislamiento de este microservicio permite contener la carga computacional derivada del procesamiento de inteligencia artificial, evitando que operaciones pesadas afecten el rendimiento general del sistema académico.

5.2.- Diseño y Modelado de la Base de Datos

La persistencia de datos en Ilerna Smart se ha diseñado siguiendo los principios fundamentales de la ingeniería de datos relacionales, garantizando la integridad referencial, la consistencia y el aislamiento de la información.

Bajo este enfoque distribuido, el dominio del sistema se descompone en esquemas independientes que limitan el contexto de persistencia a la lógica de su respectivo componente. Cada microservicio(auth-service, subject-service, document-service y test-service) administra de forma desacoplada un subconjunto de tablas especializadas.

Diagrama Entidad-Relación (ER)

El modelado lógico de las entidades se ha estructurado de forma modular y altamente relacionada. A continuación, se referencia la representación gráfica del diseño de tablas, tipos de datos, restricciones de integridad y cardinalidades que articulan el almacenamiento del ecosistema:

Cuenta con una restricción de integridad **ON DELETE SET NULL**, evitando que la baja de un docente elimine la asignatura asociada.

- **enrollments**: Gestiona el registro de matriculaciones de alumnos en las diferentes materias. Utiliza claves externas vinculadas a **users** y **subjects**, asegurando que la eliminación de un estudiante o materia limpie sus registros correspondientes en cascada (**ON DELETE CASCADE**). Posee además una restricción de unicidad compuesta **uq_enrollment** para evitar duplicidades de matrícula.

3. Tablas vinculadas a document-service:

- **topics** y **subtopics**: Organizan jerárquicamente las unidades didácticas y temarios de cada asignatura (**subjects**) mediante identificadores indexados y ordenados secuencialmente.
- **documents**: Actúa como el repositorio de metadatos de los materiales de estudio adjuntados en formato PDF por el profesorado. Almacena campos de texto extensos (**summary**) y punteros de almacenamiento (**file_url**). Sus claves foráneas apuntan a las estructuras jerárquicas con políticas de borrado controladas (**ON DELETE SET NULL**).
- **document_chunks**: Tabla especializada en el procesamiento semántico que fragmenta el texto extraído del documento original en bloques (**content**) y almacena vectores de datos mediante el tipo nativo **JSON** (**embedding**), permitiendo búsquedas conceptuales eficientes.

4. Tablas vinculadas a test-service:

- **tests** y **questions**: Modelan la evaluación automatizada generada por la API de Grok. Cada test se indexa a un documento original y contiene colecciones de preguntas (**question_text**) acompañadas de un campo explicativo de retroalimentación pedagógica (**explanation**).
- **options**: Almacena las respuestas opcionales de selección múltiple asignadas a cada pregunta. Utiliza una bandera booleana simulada (**TINYINT(1)**) para **is_correct** y se destruye en cascada si la pregunta es eliminada, blindando la consistencia del motor relacional.

- **test_submissions** y **test_answers**: Registran la evaluación continua y el desempeño transaccional del estudiante en tiempo real. Almacenan las calificaciones calculadas (**score**), las opciones seleccionadas y el estado de acierto (**is_correct**).
- **used_questions**: Tabla de control que registra un histórico hash de las preguntas ya desplegadas al usuario para evitar repeticiones en cuestionarios futuros, garantizando una experiencia evaluativa dinámica y diversificada.

Justificación de la Tercera Forma Normal (3FN)

Con el objetivo de erradicar la redundancia de datos, optimizar el uso de los bloques de almacenamiento físico del disco en el servidor y mitigar las anomalías de inserción, actualización y borrado de registros, el diseño global del script de la base de datos se estructuró bajo el cumplimiento estricto de la **Tercera Forma Normal (3FN)**:

- **Primera Forma Normal (1FN)**: Se garantiza que todas las celdas y atributos contengan exclusivamente valores únicos. Un ejemplo claro se observa en la relación entre las tablas **questions** y **options**: en lugar de almacenar las cuatro opciones de respuesta múltiple de un examen como cadenas de texto concatenadas o vectores en un único campo de la tabla de preguntas, se atomizan estructurando registros independientes en una entidad externa vinculada por su identificador.
- **Segunda Forma Normal (2FN)**: Habiendo alcanzado la 1FN, el modelo asegura que todos los atributos que no forman parte de una clave primaria dependan de manera funcional, directa y completa de dicha clave primaria, eliminando dependencias parciales. Esto es evidente en la tabla **enrollments**, donde los metadatos de la matrícula dependen exclusivamente de la clave compuesta unívoca del estudiante y la materia, sin arrastrar atributos parciales de descripción de usuario o asignatura.
- **Tercera Forma Normal (3FN)**: Finalmente, se eliminó cualquier tipo de dependencia transitiva entre los atributos no clave. Esto implica que ningún campo secundario de una tabla depende de otro atributo que tampoco sea clave primaria dentro de la misma relación. Por ejemplo, en la tabla **test_submissions**, se registran campos transaccionales fijos de auditoría rápida como **student_name** o **student_email** para congelar el estado histórico del envío, pero toda la lógica estructural, datos de perfil, contraseñas o roles no se duplican allí; permanecen estrictamente aislados en la

entidad `users` y se comunican únicamente a nivel distribuido a través de la clave primaria `student_id`.

Gracias a la estricta aplicación de la 3FN, al uso eficiente de restricciones en cascada (`CASCADE`, `SET NULL`) y al aislamiento que provee el motor relacional de MySQL bajo la arquitectura distribuida, la persistencia de *Ilerna Smart* mitiga riesgos de corrupción transaccional, reduce el espacio en disco y asegura tiempos de respuesta mínimos durante periodos de alta concurrencia evaluativa en la plataforma.

5.3 Caso de Estudio Práctico

Escenario: Evaluación Automatizada en la Asignatura "Acceso a Datos"

El siguiente caso demuestra el ciclo completo de uso de IlernaSmart, desde la carga de contenido por parte del docente hasta la retroalimentación automática al estudiante, pasando por el procesamiento inteligente del backend.

Fase 1 — Preparación (Rol Profesor, Interfaz Web React)

El docente accede a la URL `http://localhost:5174` e introduce sus credenciales. El frontend envía una petición `POST /api/auth/login` al API Gateway en el puerto 8080, que valida las credenciales contra el auth-service (puerto 8081) y devuelve un JWT firmado con el algoritmo HS256.

Una vez autenticado, el profesor navega al módulo de Asignaturas y accede a "Acceso a Datos". Desde la vista de la asignatura, despliega el Tema 3 y hace clic en el botón de subida de documento. Introduce el título "Bases de Datos NoSQL" y selecciona el PDF correspondiente.

El frontend construye un `FormData` con el título, el `subjectId`, el `topicId` y el archivo PDF, y lo envía mediante `POST /api/documents` al Gateway. El Gateway valida el JWT, extrae el rol (`TEACHER`) y redirige la petición al document-service (puerto 8083).

Fase 2 — Procesamiento Inteligente (Backend, API Groq)

El document-service recibe el PDF y ejecuta el siguiente pipeline:

Paso 1 — Almacenamiento. El archivo se guarda en el directorio local `./uploads` con un nombre UUID único para evitar colisiones.

Paso 2 — Extracción de texto. Apache PDFBox 3.x lee el archivo mediante `Loader.loadPDF()` y extrae el contenido textual completo del documento.

Paso 3 — Generación de resumen. El texto extraído se trunca a 4.000 caracteres y se envía al endpoint `POST https://api.groq.com/openai/v1/chat/completions` con el modelo `llama-3.3-70b-versatile`, temperatura 0.3 y un prompt que solicita un resumen académico en español de máximo 3 párrafos.

Paso 4 — Chunking. El texto se divide en fragmentos de aproximadamente 1.000 caracteres separando por párrafos y se almacena en la tabla `document_chunks` de MySQL. Estos chunks serán utilizados posteriormente para generar los tests.

Paso 5 — Persistencia. Se crea el registro en la tabla `documents` con el título, `subjectId`, `topicId`, la ruta del archivo y el resumen generado.

Todo el proceso queda registrado en los logs del document-service con el número de chunks generados.

Fase 3 — Evaluación (Rol Alumno, Interfaz Web React)

El estudiante Antonio David accede a IlernaSmart desde su navegador e introduce sus credenciales. El sistema le devuelve un JWT con rol `student` y le redirige a la pantalla de inicio donde aparecen sus asignaturas matriculadas.

Antonio accede a "Acceso a Datos", expande el Tema 3 y hace clic en el documento "Bases de Datos NoSQL". El DocumentViewer muestra el resumen generado por la IA colapsado con un botón "+" para expandirlo, y renderiza el PDF completo dentro de un `iframe` apuntando al endpoint público `GET /api/documents/{id}/file`.

Al finalizar la lectura, Antonio pulsa el botón flotante "Test" visible en la esquina inferior derecha de la pantalla.

El frontend envía `POST /api/tests/generate/{documentId}` al Gateway. El test-service (puerto 8084) recibe la petición y ejecuta:

1. Obtiene los chunks del documento desde el document-service mediante **GET** `/api/documents/{id}/chunks`
2. Consulta la tabla **used_questions** para recuperar los hashes SHA-256 de las preguntas ya respondidas por Antonio en tests anteriores
3. Recupera el texto de las preguntas anteriores para pasárselas a Groq como contexto
4. Envía al endpoint de Groq un prompt estructurado solicitando 10 preguntas tipo test en JSON estricto con temperatura 0.9, incluyendo la lista de preguntas a evitar
5. Parsea la respuesta JSON, filtra cualquier pregunta cuyo hash coincida con **used_questions** y guarda las preguntas y opciones en las tablas **tests**, **questions** y **options**
6. Registra los hashes de las nuevas preguntas en **used_questions**

El frontend recibe el test y presenta las preguntas de una en una, con navegación mediante botones "Anterior" y "Siguiente" y una barra de progreso en la parte superior.

Fase 4 — Feedback e Informe de Errores

Al responder la última pregunta Antonio pulsa "Enviar test". El frontend envía **POST** `/api/tests/submit` con el **testId** y el array de respuestas seleccionadas.

El test-service procesa la entrega:

1. Crea una **TestSubmission** con el **studentId**, **studentName** y **studentEmail** del alumno
2. Por cada respuesta, compara la opción seleccionada con la correcta y guarda el resultado individual en la tabla **test_answers**
3. Calcula la nota sobre 10 con la fórmula $(\text{respuestas correctas} \times 10) / \text{total preguntas}$
4. Actualiza la **TestSubmission** con la nota final y la persiste en MySQL

El frontend muestra la pantalla de resultados con la nota obtenida, por ejemplo 6/10, y el desglose pregunta por pregunta. Para cada respuesta incorrecta se muestra en rojo la opción elegida, en verde la respuesta correcta y en cursiva la explicación generada por la IA basada en el contenido del PDF original. Por ejemplo, si Antonio falló la pregunta sobre las diferencias entre MongoDB y Cassandra, el sistema le muestra exactamente por qué su respuesta era incorrecta y cuál era el concepto correcto según el material del profesor.

La calificación queda registrada en el expediente del alumno, accesible desde la sección "Calificaciones" del menú lateral, donde puede consultar su historial de intentos, la media por documento y la nota más alta obtenida.

El profesor, accediendo a la misma sección desde su cuenta, puede ver el listado de todos los alumnos que han realizado el test del Tema 3, consultar el detalle de las respuestas de cada uno y modificar manualmente cualquier nota si lo considera necesario.